

Java 面试 Q&A 整理

合并说明：本文件已并入 JVM.md、interview.txt（朋友面经）、java_project_qa.md 的独特内容，分别见第十四~十七节。

一、Java 核心基础

Q1: HashMap 扩容原理

1.7 vs 1.8 区别：

版本	数据结构	扩容时机	扩容方式	扩容大小
1.7	数组+链表	$size > threshold$ && 当前桶非空	头插法（多线程死循环）	2倍
1.8	数组+链表+红黑树	$size > threshold$	尾插法	2倍

1.8 扩容过程：

1. 当 $size > loadFactor * capacity$ （默认 0.75）时触发扩容
2. 容量变为原来的 2 倍
3. 重新计算每个元素的位置——不需要重新 hash，只需判断原 hash 值新增 bit 是 0 还是 1：
 - 0 → 原索引不变
 - 1 → 原索引 + oldCap
4. 链表长度 ≥ 8 且数组长度 ≥ 64 时，链表转红黑树
 1. 某个桶达到 8 时：
 1. 数组长度小于 64，只扩容，不树化
 2. 大于 64，转为红黑树
5. 扩容后重新分配时，红黑树节点 ≤ 6 时退化为链表

扩容过程：

假设容量从 4 扩容到 8，某个 key 的 hash 值二进制为 $\dots 0101$ ：

- 旧索引 = $0101 \& 0011 = 0001$ （即 1）
- 判断 $hash \& oldCap = 0101 \& 0100 = 0100$ （不等于 0）
- 新索引 = 原索引 + oldCap = $1 + 4 = 5$

如果另一个 key 的 hash 为 $\dots 0001$ ：

- $hash \& oldCap = 0 \rightarrow$ 新索引 = 原索引 = 1。

追问：1000 条数据往初始容量 16 的 HashMap 放，会扩容几次？

- 扩容阈值 = $16 \times 0.75 = 12$
- 扩容路径：16 → 32（12 满）→ 64（24 满）→ 128（48 满）→ 256（96 满）→ 512（192 满）→ 1024（384 满）
- 1000 条数据落在 1024 容量内，共扩容 6 次

延伸：

- 为什么负载因子是 0.75？空间和时间的折中，太低浪费内存，太高碰撞概率大
- 为什么容量是 2 的幂？方便位运算 $(n-1) \& \text{hash}$ 代替取模
- ConcurrentHashMap 扩容：多线程协同迁移，每个线程负责一段槽位

Q2: String 底层实现

- JDK 8 及之前：char[] 数组，每个 char 占 2 字节
- JDK 9+：byte[] + coder 标记，Latin-1 字符用 1 字节，节省内存（Compact Strings）
- jdk9：coder 区分 Latin-1 和 utf-16

```
// "hello" 和 "he" + new String("llo") 比较
String a = "hello";           // 字符串常量池
String b = "he" + new String("llo"); // 运行时拼接, new String在堆中
// a == b → false (a在常量池, b在堆中)
// a.equals(b) → true
```

延伸——字符串拼接原理：

- 编译期常量拼接 → 编译器优化为常量（"a"+"b" 直接编译为 "ab"）
- 变量拼接 → 使用 StringBuilder.append() 或 StringConcatFactory
- 循环中拼接：务必显式使用 StringBuilder，否则每次循环创建新实例

Q3: int vs Integer 比较（拆箱与装箱）

```
int a = 100;
Integer b = 100;
// a == b → true (Integer自动拆箱为int, 值比较)

Integer c = 200;
Integer d = 200;
// c == d → false (超出缓存池 -128~127, 各自new对象)
// 默认缓存上限可通过 -XX:AutoBoxCacheMax=size 调整
```

IntegerCache 机制： Integer 内部类 IntegerCache，默认缓存 [-128, 127] 的 Integer 对象。valueOf() 方法在范围内返回缓存对象，超出范围 new 新对象。Byte/Short/Long 也有类似缓存，范围固定 [-128, 127]。为什么 Integer 要做 IntegerCache？：目的是节省内存、提升性能。范围内这些数使用频率高。避免使用 new Integer 创建，使用 Integer i = 10; 这种调用 valueOf() 自动装箱。

Q4: 多线程 run() vs start()

```
new Thread(() -> System.out.println("A")).run(); // 当前线程执行，同步
new Thread(() -> System.out.println("B")).start(); // 新线程执行，异步
```

- run()：当前线程直接调用方法体，不会创建新线程
- start()：JVM 创建新线程，新线程调用 run()。一个线程只能 start 一次，再次调用抛 IllegalStateException

延伸——线程生命周期（6 种状态）： NEW → RUNNABLE → BLOCKED / WAITING / TIMED_WAITING → TERMINATED

Q5: 单例模式手写

```
// DCL (双重检查锁) —最经典的线程安全懒汉式
public class Singleton {
    // volatile 防止指令重排 (分配内存 → 初始化 → 赋值引用, 可能乱序)
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) { // 第一重检查 (避免加锁开销)
            synchronized (Singleton.class) {
                if (instance == null) { // 第二重检查 (防止重复创建)
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

延伸:

- 饿汉式: 类加载时初始化, 线程安全但可能浪费内存
- 枚举式: 天然防反射、防序列化攻击 (Effective Java 推荐)
- Spring 单例 Bean 默认是单例, 但 Spring 管理的"单例"是容器级别的 (一个 BeanDefinition 一个实例)

Q6: 集合数据去重方法

场景	方法
List 去重	<code>new ArrayList<>(new HashSet<>(list))</code> (无序); <code>ls.stream().distinct().toList()</code>
List 去重保持顺序	<code>new ArrayList<>(new LinkedHashSet<>(list))</code>
按对象某个字段去重	<code>list.stream().filter(distinctByKey(User::getId)).collect(toList())</code>
自定义去重 (TreeSet)	<code>new TreeSet<>(Comparator.comparing(User::getId))</code>

```
// 按字段去重的工具方法
private static <T> Predicate<T> distinctByKey(Function<? super T, ?> keyExtractor) {
    Set<Object> seen = ConcurrentHashMap.newKeySet();
    return t -> seen.add(keyExtractor.apply(t));
}
```

Q7: List 转 Map

```
// 基本转换
Map<Long, User> map = list.stream()
    .collect(Collectors.toMap(User::getId, Function.identity()));
```

```
// key 冲突处理：保留旧值
Map<Long, User> map2 = list.stream()
    .collect(Collectors.toMap(User::getId, u -> u, (old, nu) -> old));

// 按字段分组
Map<Long, List<User>> group = list.stream()
    .collect(Collectors.groupingBy(User::getDeptId));
```

注意：Collectors.toMap() key 重复默认抛 IllegalStateException，必须指定 mergeFunction。

Q8: 多线程如何保证线程安全

线程安全问题本质是多个线程同时访问共享资源导致的数据不一致。

可以根据实际场景，按下面的思路来选择方案：

1. 能不共享吗？ -> 优先使用局部变量。
2. 能直接使用现成的安全类吗？ -> 使用 ConcurrentHashMap, Atomic* 等。
3. 必须自己控制锁吗？ -> 优先使用 synchronized，代码更简单；需要超时、中断等高级功能时，再考虑 ReentrantLock。
4. 是想“隔离”而非“共享”吗？ -> 使用 ThreadLocal。

任何同步机制都会带来或多或少的性能开销，基本原则是：只在必要时保护数据，尽量缩小同步代码块的范围。

方式	说明
synchronized	JVM 内置锁，自动加/解锁，悲观锁
ReentrantLock（可重入锁）	API 层面，支持公平锁、条件变量、可中断
volatile	保证可见性+禁止指令重排，不保证原子性
Atomic 类	CAS 无锁，适合计数/状态位
ThreadLocal	线程私有副本，空间换安全;使用完必须手动remove()
不可变对象	final 修饰，天然线程安全
并发集合	ConcurrentHashMap、CopyOnWriteArrayList

线程生命周期：

将线程的生命周期与生活中的场景关联起来，会更容易记忆：

- **NEW**：你刚拿到一张招聘启事。
- **RUNNABLE**：你投了简历，正在等待HR筛选（就绪）或正在面试（运行）。
- **BLOCKED**：会议室（锁）被占用，你被挡在门外等别人出来。
- **WAITING**：面试结束了，你回家等通知（无限期等待），直到HR给你打电话（notify）。
- **TIMED_WAITING**：HR让你等10分钟，她去找主管确认（超时等待）。
- **TERMINATED**：你收到offer并入职了；或者被明确告知不合适，流程结束。

Q9: 线程池核心线程数如何设计

公式（《Java 并发编程实战》）：

任务类型	公式
CPU 密集型(加密，排序等)	$N_CPU + 1$

任务类型

公式

IO 密集型（网络调用、数据库读写、文件 IO）

$N_CPU * 2$ 或 $N_CPU / (1 - \text{阻塞系数})$ ，阻塞系数 $\approx 0.8 \sim 0.9$

混合型

拆分为 CPU 密集 + IO 密集两个线程池

实际调整因子：

- 考虑其他服务共用机器 → 适当降低
- 考虑任务平均耗时、峰值 QPS → $\text{coreSize} = \text{QPS} \times \text{avgRT} / 1000$
- 核心线程数 = 期望 QPS × 单任务耗时（秒）

```
// 示例：期望 QPS=200，单个任务 50ms  
// coreSize = 200 × 0.05 = 10
```

延伸——线程池参数拒绝策略：

- AbortPolicy（默认）：抛异常
- CallerRunsPolicy：由调用线程执行
- DiscardPolicy：静默丢弃
- DiscardOldestPolicy：丢弃队列中最旧任务

****动态线程池（Dynamic Thread Pool）：****核心思路是把线程池关键参数外置，并支持运行时调整：

1. **不要写死参数：**把 `corePoolSize` 和 `maximumPoolSize` 丢到配置中心（比如 Nacos、Apollo）。
2. **利用 Java 原生 API 动态修改：**`ThreadPoolExecutor` 其实自带了 `setCorePoolSize(int)` 和 `setMaximumPoolSize(int)` 方法！
3. **监控 + 联动：**写个定时任务或者结合监控系统，一旦发现线程池的队列堆积率超过 80%，或者活跃线程数长期触顶，直接在配置中心把核心线程数调大，**不需要重启服务器，线上秒级生效！**

4.

```
// 不能在运行时直接修改 corePoolSize，但可以 set  
executor.setCorePoolSize(newCoreSize);  
executor.setMaximumPoolSize(newMaxSize);
```

Q10: 死锁怎么造成？如何避免？

四个必要条件（缺一不可）：

1. 互斥：资源独占
2. 持有并等待：持有锁 A 等锁 B
3. 不可剥夺：不能强行抢锁
4. 循环等待：A→B→C→A 的等待环

避免方法：

- **破坏循环等待：**统一加锁顺序，按固定顺序获取锁；比如，根据账户的 ID 大小来加锁，永远先锁 ID 小的，再锁 ID 大的！
- **破坏持有并等待：**一次性申请所有锁（如 `tryLock` 超时）；
- **破坏不可剥夺：**使用 `ReentrantLock.tryLock(timeout, unit)`
- **检测工具：**jstack 查看线程状态，查找 BLOCKED 和 waiting to lock
- **数据库死锁：**缩短事务、固定访问顺序、合理索引减少锁范围
- 能用局部变量的就不要使用共享成员变量。

- 能用无锁原子类（如 `AtomicInteger`）的就别用 `synchronized`。
- 如果必须加锁，尽量缩小锁范围，避免长时间持有锁。

Q11: `LocalDateTime.now()` 线程安全吗？

线程安全。与 `SimpleDateFormat`（非线程安全）不同：

- `LocalDateTime`、`LocalDate`、`Instant` 等 Java 8 时间类都是**不可变对象**，天然线程安全
- `DateTimeFormatter` 也是不可变+线程安全的
- `SimpleDateFormat` → 必须用 `ThreadLocal` 包装或加锁
- `Date` 可变的 `setTime()` → 不安全

延伸：导出文件名用

`LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMddHHmmss"))` 是安全的。

二、Spring 框架

Q12: Spring Bean 是否线程安全？

默认不保证线程安全。

- Spring 不提供线程安全策略，线程安全性由 Bean 自身的设计决定
- **无状态 Bean**（如 `Service/Controller` 中不存成员变量）：天然线程安全
- **有状态 Bean**（持有可变成员变量）：非线程安全，需自行控制
- **prototype 作用域**：每次创建新实例，不等于线程安全，只是减少了共享
- **@Scope("request")**：每个请求一个实例，实际上是 `ThreadLocal` 实现

最佳实践：Controller/Service 层保持无状态，必要状态数据存入 `ThreadLocal` 或 `Redis`。

Q13: @Transactional 失效的三大原因

1. 方法不是 public

- Spring AOP 基于动态代理（JDK/CGLIB），只能代理 `public` 方法
- 私有/受保护方法的 `@Transactional` 直接无效

2. 同类自调用

```
@Service
public class UserService {
    public void methodA() { // 外部调用，有事务
        this.methodB();    // this 是原始对象，不是代理，事务不生效！
    }

    @Transactional
    public void methodB() { }
}
```

解决方案：注入自己（`@Autowired private UserService self`），通过 `self.methodB()` 调用。

3. 异常被捕获

```

@Transactional
public void doSomething() {
    try {
        dbOperation();        // 抛异常
    } catch (Exception e) {
        log.error("error", e); // 事务不回滚！
    }
}

```

- `@Transactional` 默认回滚策略：`RuntimeException` 和 `Error` 回滚，受检异常不回滚
- 指定回滚：`@Transactional(rollbackFor = Exception.class)`
- 异常被 `catch` 吞掉后 Spring 感知不到，自然不会回滚

延伸： `@Transactional` 的 `propagation` 传播行为 —— `REQUIRED`（默认）、`REQUIRES_NEW`、`NESTED` 等。

Q14: Spring 事务执行机制

1. **代理创建：** Spring 为 `@Transactional` 的 Bean 生成 AOP 代理对象（proxy）
2. **获取连接：** 代理拦截方法调用 → 从 `DataSource` 获取数据库连接 → 关闭自动提交（`setAutoCommit(false)`）
3. **绑定连接：** 当前连接绑定到 `ThreadLocal`（`TransactionSynchronizationManager`）
4. **执行业务：** 同一个事务内所有 SQL 共用同一条连接
5. **提交/回滚：** 正常完成 → `commit`；异常判断 `rollbackFor` → `rollback`
6. **释放连接：** 解绑 `ThreadLocal`，归还连接池

关键： 同一个事务内多次数据库操作使用的必须是同一个 `Connection`（`ThreadLocal` 保证）。

spring事务三个核心：

TransactionInterceptor（事务拦截器）： 事务 AOP 的核心拦截器，负责拦截带有 `@Transactional` 的方法调用，并在方法执行前后织入事务逻辑。

PlatformTransactionManager（事务管理器）： 事务管理的统一接口，例如 `JDBC/MyBatis` 常用 `DataSourceTransactionManager`。它负责获取连接、开启事务、提交事务和回滚事务。

TransactionSynchronizationManager（事务同步管理器）： 底层基于 `ThreadLocal` 保存当前线程的事务资源。它负责把数据库连接绑定到当前线程，保证同一个事务中的多个 `Mapper` 使用同一个连接。

Q15: Spring MVC 和 Spring Boot 区别

维度	Spring MVC	Spring Boot
定位	Web 框架（MVC 实现）	快速开发脚手架
配置	大量 XML/Java 配置	自动配置 + Starter
部署	需要外部 Servlet 容器（Tomcat）	内嵌 Tomcat，java -jar 运行
依赖管理	手动管理版本	版本仲裁（spring-boot-dependencies）
关注点	只做 MVC	基础设施全家桶（监控/健康检查/外部化配置）
关系	Spring Boot 内置了 Spring MVC	Spring Boot = Spring MVC + 自动配置 + Starter + Actuator

延伸——内嵌 Tomcat 原理： Spring Boot 启动时创建 `TomcatServletWebServerFactory` → `new Tomcat` 实例 → 注册 `DispatcherServlet` → 启动监听端口。jar 包之所以是 web 应用，是因为内嵌了 Servlet 容器。

Q16: @SpringBootApplication 注解

```
@SpringBootApplication // 三合一注解
├─ @SpringBootConfiguration // = @Configuration, Java 配置类
├─ @EnableAutoConfiguration // 自动配置, 通过 spring.factories 加载
xxxAutoConfiguration
├─ @ComponentScan           // 扫描当前包及子包下的组件
```

自动配置原理:

1. @EnableAutoConfiguration → @Import(AutoConfigurationImportSelector.class)
2. 读取 META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
3. 按条件注解 (@ConditionalOnClass、@ConditionalOnBean、@ConditionalOnMissingBean 等) 过滤
4. 符合条件则创建 Bean 注入容器

Q17: Spring Security 认证授权

核心流程:

1. 认证 (Authentication) : 用户 → Filter Chain → AuthenticationManager → ProviderManager → UserDetailsService 查库 → 比对密码 (BCryptPasswordEncoder) → 返回 Authentication 对象存入 SecurityContextHolder
2. 授权 (Authorization) : 请求 → FilterSecurityInterceptor → 比对所需角色/权限 → 通过则放行

核心组件:

- SecurityContextHolder: 通过 ThreadLocal 持有当前用户信息
- BCryptPasswordEncoder: 加盐哈希, 每次加密生成随机盐, 结果不同
- JWT + Spring Security: 自定义 OncePerRequestFilter 解析 Token → 构建 Authentication → 存入 SecurityContextHolder

```
// BCryptPasswordEncoder 特点: 同一密码每次加密结果不同 (随机盐)
String hash1 = encoder.encode("123456"); // $2a$10$xxx1
String hash2 = encoder.encode("123456"); // $2a$10$xxx2 (不同!)
// matches 方法解析盐值后比对
encoder.matches("123456", hash1); // true
```

三、Spring Cloud 微服务

Q18: OpenFeign 跨服务调用全链路

注解使用:

```
// 主类
@EnableFeignClients
@SpringBootApplication
public class Application {}

// 接口定义
@FeignClient(name = "order-service", path = "/api/orders",
```



```

        fallbackFactory = OrderClientFallbackFactory.class) // 配置了fallback, 这个
服务宕机就走fallback的实现
    public interface OrderClient {
        @GetMapping("/{id}")
        Result<Order> getById(@PathVariable Long id);

        @PostMapping
        Result<Void> create(@RequestBody OrderDTO dto);
    }

```

底层调用逻辑：

1. @EnableFeignClients 扫描 @FeignClient 接口
2. JDK 动态代理生成接口实现类
3. 调用方法时 → MethodHandler 构建 RequestTemplate (URL/Method/Headers/Body)
4. LoadBalancer 从 Nacos 获取服务列表，负载均衡选择实例
5. 实际 HTTP：默认 HttpURLConnection，推荐替换为 Apache HttpClient (连接池)
6. 响应解码器 → 转换回方法返回类型
7. 熔断：Sentinel/Hystrix 保护，降级走 fallback

Feign 调用失败排查：

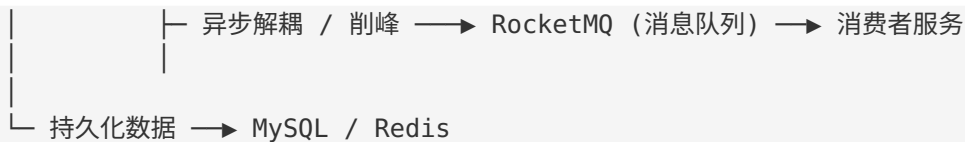
- 检查 Nacos 服务是否在线
- 检查 @FeignClient name 是否与服务名一致
- 检查超时配置 (feign.client.config.xxx.connectTimeout/readTimeout)
- 开启 Feign 日志 logging.level.com.xxx.client.OrderClient=DEBUG
- 使用 Arthas trace 追踪完整调用链

feign 每次发 http 请求建立连接问题： 配置 Apache HttpClient 连接池，复用 TCP 连接

springcloudealibaba

核心能力	Netflix (已死或停更)	Spring Cloud Alibaba (替代者)	一句话优势
服务注册发现	Eureka (停更)	Nacos	支持AP/CP切换，健康检查更灵敏，自带配置中心
配置管理	Config + Bus (难用)	Nacos Config	实时热更新，灰度发布，权限控制，一个平台管所有
限流熔断	Hystrix (停更)	Sentinel	可视化Dashboard，动态规则调整，支持热点参数限流





完整流程应该是这样的：

1. **Gateway 生成内部 Token**：用一把**所有微服务共享的密钥**签名，有效期设长。
2. **服务 A 携带 Token 调服务 B**：通过 Feign 拦截器自动带上。
3. **服务 B 收到请求，用同样的密钥验证 Token 签名**：确认它是 Gateway 签发的，不是伪造的。
4. **服务 B 解析出用户身份**：设置到自己的 SecurityContext 里，然后做权限判断。

同步调用传Token，异步（MQ）消息传身份

1. 用户请求到达 Gateway
2. Gateway 校验用户JWT（验签名、过期、黑名单）
3. Gateway 向 Nacos 查询服务A的地址
4. Gateway 生成内部Token（长有效期），放入 X-Internal-Token 头
5. Gateway 转发请求到服务A
6. 服务A 校验内部Token（验签名），解析出 userId/roles 设置到 SecurityContext
7. 服务A 执行业务逻辑
8. 如果服务A需要调用服务B：
 - 同步调用：Feign拦截器自动携带 X-Internal-Token → 服务B验签
 - 异步调用：发MQ消息，消息体里携带 userId → 服务B从消息体取身份

Q19: 微服务体系核心组件

组件	功能	选型
注册中心	服务注册/发现	Nacos / Eureka
配置中心	动态配置	Nacos Config
网关	统一入口	Spring Cloud Gateway
远程调用	服务间通信	OpenFeign
熔断降级	防止级联故障	Sentinel
负载均衡	请求分发	Spring Cloud LoadBalancer

四、MySQL 数据库

MVCC：

让读不阻塞写，写不阻塞读，还能保证读到的数据是一致的历史快照。

隐藏列：

隐藏列	全名	作用
DB_TRX_ID	事务ID	最后一次修改这行数据的事务ID
DB_ROLL_PTR	回滚指针	指向Undo Log中这行数据的上一个版本
DB_ROW_ID	行ID（如果没有主键）	自增行号

Undo Log:

每当你更新一行数据，InnoDB不直接覆盖，而是：

- 1. 把旧数据拷贝到Undo Log。
- 2. 更新当前数据，把 DB_TRX_ID 设成当前事务ID。
- 3. 把 DB_ROLL_PTR 指向Undo Log里的旧版本。

这样一行数据就形成了一个版本链：

当前版本 (TRX_ID=100) → Undo Log v2 (TRX_ID=99) → Undo Log v1 (TRX_ID=88) → 初始版本

ReadView:

当你启动一个事务执行快照读（普通的SELECT），InnoDB会生成一个ReadView，它包含：

- **m_ids**：当前活跃的（未提交的）事务ID列表。
- **min_trx_id**：活跃事务的最小ID。
- **max_trx_id**：下一个即将分配的事务ID。
- **creator_trx_id**：创建这个ReadView的事务ID。

ReadView就是个“时间滤镜”，它让你只能看到在你事务开始前已经提交的数据。

```
if (TRX_ID == creator_trx_id) {
    // 我自己修改的，可见
    return 可见;
}
if (TRX_ID < min_trx_id) {
    // 在我开始前就提交了，可见
    return 可见;
}
if (TRX_ID >= max_trx_id) {
    // 是我开始之后才启动的事务，不可见
    return 不可见，沿着版本链往下找;
}
if (TRX_ID 在 m_ids 里) {
    // 是活跃事务，还没提交，不可见
    return 不可见，往下找;
}
// 否则，已经提交了，可见
return 可见;
```

Q20: MySQL 底层 B+ 树

为什么用 B+ 树而不是 B 树/红黑树/哈希？

对比	B+ 树	B 树	红黑树	哈希
数据存储	仅在叶子节点	非叶节点也存	节点存单值	数组+链表
叶子节点	双向链表连接	无连接	-	-
范围查询	极快（遍历链表）	需中序遍历	需遍历	不支持
高度	矮（一个节点存16KB）	矮	高	-
IO 次数	少	少	多	一次

B+ 树结构：

- 非叶子节点只存 key + 子节点指针（16KB 一页可存约 1200 个索引键）
- 叶子节点存完整行数据（聚簇索引）或主键值（二级索引）
- 叶子节点之间用双向指针链接，支持正序/倒序扫描

Q21: 回表是什么？

回表：通过二级索引查询时，先在二级索引 B+ 树找到主键值，再回到聚簇索引 B+ 树查完整行数据的过程。

```
查询：SELECT * FROM user WHERE name = '张三'; -- name 有索引
```

Step 1: 走 name 索引 → 叶子节点找到 name='张三' 对应的主键 id=100

Step 2: 拿 id=100 回聚簇索引 → 找到完整行数据 ← 这就是回表

优化——覆盖索引：如果查询的列都在二级索引中，直接返回，不回表。

```
CREATE INDEX idx_name_age ON user(name, age);
SELECT name, age FROM user WHERE name = '张三'; -- 覆盖索引，不回表
```

Q22: 联合索引与最左匹配原则

最左匹配：联合索引 (a, b, c) 按 a → b → c 的顺序建立 B+ 树，查询条件必须从最左列开始才能走索引。

联合索引：idx_abc(a, b, c)

WHERE a=1	→ 走索引 (a)
WHERE a=1 AND b=2	→ 走索引 (a+b)
WHERE a=1 AND b=2 AND c=3	→ 走索引 (完整)
WHERE a=1 AND c=3	→ a 走索引, c 不走 (跳过 b)
WHERE b=2	→ 不走索引 (缺 a)
WHERE b=2 AND c=3	→ 不走索引 (缺 a)
WHERE a=1 AND b>2 AND c=3	→ a+b 走索引, c 不走 (范围后的列失效)
WHERE a>20 AND b=1 AND c=2	→ 只有 a 走索引 (a 是范围查询, b/c 失效)

追问：【联合索引 idx(a, b, c)，SQL = WHERE a>20 AND b=1 ORDER BY c DESC 索引怎么走？】

- **过滤阶段：**a 范围查询 → 只用到 a (b=1 是等值但 a 已是范围，所以 b 不能用于过滤)
- **排序阶段：**因为 a 是范围查询，跳过了 b，索引 (a, c) 不是有序的，排序也用不上索引
- **结论：**仅 a 走索引过滤，ORDER BY c 走 filesort

IN 导致索引失效的情况：

- MySQL 5.5 之前 IN 完全不走索引
- MySQL 5.6+ IN 走索引，但条件是优化器根据数据分布（基数）决定的。大量 IN 值 → 优化器可能认为全表扫描更快 → 强制索引用 `FORCE INDEX`

Q23: INT(10) 和 INT(11) 的区别

核心结论：没有任何区别。

- 括号里的数字是**显示宽度**，不是存储大小

- INT 固定 4 字节存储, 范围 -2147483648 ~ 2147483647
- 显示宽度只在 **ZEROFILL** 时生效: `INT(3) ZEROFILL` 插入 1 → 显示 001
- MySQL 8.0.17 开始 display width 已废弃

INT(1) **INT(5)** **INT(10)** **INT(11)**

最大值相同: 2147483647 相同 相同 相同

Q24: ORDER BY 什么情况索引失效?

1. 排序字段不在索引中 → filesort
2. 排序字段顺序与索引不匹配 (如索引 (a,b), ORDER BY b) → filesort
3. 升序降序混用与索引排序不一致 → filesort
4. 排序带有非索引列的函数/表达式 → filesort
5. 走了一个索引, ORDER BY 用了另一个索引列 → filesort (MySQL 只会选一个索引)
6. WHERE 是范围查询, ORDER BY 不在同一索引中 → filesort

Q25: SQL 数据量大、有索引还是很慢怎么办?

1. 确认索引真正命中: EXPLAIN 看 type (ALL/INDEX 说明没走或全索引扫)
2. 选错索引: EXPLAIN 看 possible_keys 和 key 不一致 → FORCE INDEX
3. 回表太大: 建覆盖索引消除回表
4. limit 深翻页: 用"游标法"替代 offset → WHERE id > last_id LIMIT n
5. 数据量确实过大 (千万级): 分库分表 (ShardingSphere)、引入 ES、冷热分离
6. 慢查询日志: slow_query_log + pt-query-digest 分析
7. 锁竞争: SHOW ENGINE INNODB STATUS 看是否有锁等待

数据量大, 有索引还很慢, 按这个顺序排雷:

1. 查执行计划 (EXPLAIN): 看 type, key, Extra. 扫一眼有没有索引失效、没用上覆盖索引? (八股文基本功)
2. 查实际扫描行数 (EXPLAIN ANALYZE): 优化器预估的行数和实际差别大吗? (MySQL 8.0+ 神器)
3. 查锁与事务: 是不是被别的会话阻塞了? (信息_schema)
4. 查硬件和配置: Buffer Pool 是不是太小? 磁盘是不是瓶颈?
5. 查SQL本身: 是不是深度分页? 是不是 SELECT * 一把梭?
6. 灵魂拷问: 这个需求, 适合用MySQL吗?

带头大哥不能死, 中间兄弟不能断 列上有函数, 索引变废物 类型不对, 全表遭罪 Like一开头, 索引必定丢 范围一出现, 右列全不见 非此即彼, 索引难起 OR两边, 都得是索引, 否则全表扫描安排

看SQL先EXPLAIN, type字段定乾坤。最少要到range级, 看到ALL就发晕。key_len量用量, 最左前缀断没断。extra里找祸根, filesort和temp最烦人。rows虽然不准确, 数量级上见真章。

Q26: 千万级数据查询优化

策略	说明
分库分表	水平拆分, ShardingSphere / MyCat
读写分离	主库写入, 从库查询
ES 同步	Canal 监听 binlog 同步到 ES, 搜索走 ES
冷热分离	近期数据在 MySQL, 历史数据归档到 Hive/TiDB
分区表	MySQL 原生 partition, 按时间范围分区
只查必要列	SELECT col1,col2 而非 SELECT *

策略	说明
缓存预热	Redis 缓存热点数据

Q27: 分库分表

分库分表方式：

- **垂直分库**：按业务拆分（用户库、订单库、商品库）
- **垂直分表**：按字段拆分（主表存常用字段，扩展表存大字段）
- **水平分库**：按分片键 hash 分到不同库
- **水平分表**：同一库内按规则拆分为多个表

分片键选择原则：选择最频繁的查询条件（如 userId、orderId），避免跨库 join/事务。

带来的问题：

- 分布式 ID 生成（雪花算法）
- 跨库 join → 应用层聚合或冗余存储
- 跨库事务 → 柔性事务（Seata/可靠消息）
- 扩容 → 一致性哈希、双倍扩容

乐观锁 悲观锁

悲观锁：每次拿数据时都认为**别人会修改**，所以全程加锁，不让别人碰。

```
-- 开启事务
BEGIN;

-- 查库存，加排他锁（其他事务不能读也不能写这一行）
SELECT stock FROM product WHERE id = 1 FOR UPDATE;

-- 你拿到stock=10，开始扣减
UPDATE product SET stock = stock - 1 WHERE id = 1;

-- 提交事务，释放锁
COMMIT;
```

SELECT ... FOR UPDATE 会给这行加**行级排他锁**。其他事务想读这行（除非用 快照读 绕过）或更新，都得等待当前事务释放锁。

乐观锁：每次拿数据时都认为**别人不会修改**，只在最后更新时检查一下数据有没有被人动过。

依赖一个 `version` 字段。

```
-- 查询当前库存和版本号
SELECT stock, version FROM product WHERE id = 1;
-- 假设查到 stock=10, version=1

-- 更新时带上版本号条件
UPDATE product
SET stock = stock - 1, version = version + 1
WHERE id = 1 AND version = 1;
```

关键点：如果 WHERE version = 1 影响行数为0，说明在你操作期间有人改过，版本号从1变成了2。你需要重新查、重新算。

```

// 实体类加@Version注解
@Data
public class Product {
    private Long id;
    private String name;
    private Integer stock;
    @Version
    private Integer version;
}

// 配置插件 MP可以自动实现
@Configuration
public class MybatisPlusConfig {
    @Bean
    public MybatisPlusInterceptor mybatisPlusInterceptor() {
        MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();
        interceptor.addInnerInterceptor(new OptimisticLockerInnerInterceptor());
        return interceptor;
    }
}

// 业务代码
public void deductStock(Long productId) {
    Product product = productMapper.selectById(productId);
    product.setStock(product.getStock() - 1);
    int rows = productMapper.updateById(product);
    if (rows == 0) {
        throw new RetryException("数据已被修改，请重试");
    }
}

```

选型：

有高并发写冲突吗？

- └─ 没有（或者冲突很低）
 - └─ 乐观锁，省心省力
- └─ 有，但冲突集中在少数热点（如秒杀单品）
 - └─ 悲观锁 + 排队（或者直接用Redis原子扣减）
- └─ 有，且读多写少（如个人资料编辑）
 - └─ 乐观锁，冲突概率极低
- └─ 有，且是长事务（跨服务调用）
 - └─ 乐观锁（悲观锁长事务会导致锁一直不释放，系统卡死）

令牌桶：

令牌桶算法的三个核心参数：

1. **令牌生成速率**：你爸多久放一个硬币。（比如每秒10个，即10 QPS）
2. **桶容量**：存钱罐最多装几个硬币。（比如100个，应对突发流量）
3. **每次请求消耗令牌数**：一般每次消耗1个。

漏桶和令牌桶：

- **漏桶**：像个有洞的水桶，水（请求）从上面倒进去，从底部匀速流出。**不管你倒多快，出水速度恒定。**相当于强制**削峰填谷**，把流量整得平平的。但它的问题是一旦桶满了，多余的水直接溢出（拒绝请求），**完全没有突发处理能力**。
- **令牌桶**：令牌以恒定速率产生，但可以**攒着**。平时没人用，令牌就攒满了一桶（比如100个）。突然来了100个请求，只要桶里有100个令牌，就能全部放行。**它允许一定程度的突发流量，这是它最大的优势。**

实现：

1. 单机：Guava (google)
2. 分布式：Redisson RRateLimiter

```
RRateLimiter limiter = redisson.getRateLimiter("buyLimiter");
// 初始化，设置模式为“预先生成令牌”，总量10个，每秒产生5个
limiter.trySetRate(RateType.OVERALL, 10, 5, RateIntervalUnit.SECONDS);

// 尝试获取1个令牌，最多等待2秒
if (limiter.tryAcquire(1, 2, TimeUnit.SECONDS)) {
    doBusiness();
}
```

3. Redis + Lua

```
-- 获取令牌的Lua脚本
local key = KEYS[1] -- 令牌桶的key，如 "rate:buy"
local rate = tonumber(ARGV[1]) -- 每秒放几个令牌
local capacity = tonumber(ARGV[2]) -- 桶容量
local now = tonumber(ARGV[3]) -- 当前时间戳（毫秒）
local requested = tonumber(ARGV[4]) -- 请求要几个令牌，一般是1

-- 计算从上次补充到现在的时间差
local last_refresh = tonumber(redis.call('get', key .. ':ts')) or now
local tokens = tonumber(redis.call('get', key .. ':tokens')) or capacity

-- 补充令牌：时间差(秒) * 速率 + 当前令牌数，但不能超过桶容量
local delta = math.max(0, now - last_refresh) / 1000 * rate
tokens = math.min(capacity, tokens + delta)

-- 判断是否够
if tokens >= requested then
    tokens = tokens - requested
    -- 更新令牌数和时间戳
    redis.call('set', key .. ':tokens', tokens)
    redis.call('set', key .. ':ts', now)
    return 1 -- 拿到令牌
else
    return 0 -- 不够
end
```

```
public boolean acquireToken(String key, double rate, int capacity) {
    DefaultRedisScript<Long> script = new DefaultRedisScript<>();
    script.setScriptSource(new ResourceScriptSource(new
ClassPathResource("rate_limiter.lua")));
    script.setResultType(Long.class);
}
```

```

        Long result = redisTemplate.execute(script,
            Arrays.asList(key),
            String.valueOf(rate), String.valueOf(capacity),
            String.valueOf(System.currentTimeMillis()), "1");
        return result != null && result == 1;
    }

```

五、Redis 缓存

Q28: Redisson 分布式锁 + 看门狗机制

基本用法：

```

RLock lock = redissonClient.getLock("lock:report:" + reportId);
lock.lock(); // 不传过期时间 → 看门狗启动
try {
    generateReport(); // 耗时不定的业务
} finally {
    lock.unlock(); // finally 释放，同时停止看门狗
}

```

看门狗原理：

- 默认过期时间 30s，每 10s (=30s/3) 续期一次
- 续期通过 Netty 定时任务执行 Lua 脚本 pexpire
- 续期前检查 EXPIRATION_RENEWAL_MAP 中锁是否还存在（unlock 时会 remove）
- 触发条件：lock() 不传 leaseTime 或 传 -1
- 传了具体 leaseTime → 不启动看门狗，到期即删

网络问题导致看门狗失效：

- 场景：GC 停顿或网络分区 → 续期指令发不到 Redis → key 30s 过期 → 其他线程拿到锁
- 影响：出现两个线程同时持有“锁”，数据不一致
- 缓解：
 - 适当增大 lockWatchdogTimeout（配置类设置）
 - 业务层加乐观锁/版本号做二次校验
 - 使用 Redisson 的红锁（RedLock）多节点保证（生产环境较少使用）

Redisson vs Jedis vs Lettuce： | 特性 | Jedis | Lettuce | Redisson | |-----|-----|-----|-----| | 连接方式 | 同步 | 异步 (Netty) | 异步 (Netty) | | API 风格 | Redis 原生命令 | 原生命令 | 高级 Java 对象 (Lock/Map/Queue) | | 分布式锁 | 需自实现 | 需自实现 | 内置看门狗 + 可重入 |

Q29: Redisson 实现报告生成功能（场景题）

场景：前端点击“生成报告”，后端异步生成，防止重复提交。

```

用户 → 前端点“生成报告”
      → 后端 GET /report/generate/{id}
      → Redis SETNX "lock:report:{id}" （1分钟内同一报告不能重复生成）
      → 成功：提交 XXL-Job 异步任务
      → 返回 “报告生成中，请稍后刷新”

```

- 前端轮询 GET /report/status/{id} (每 3s 一次)
- 生成完成 → 返回下载链接

```
public Result generateReport(Long reportId) {
    RLock lock = redissonClient.getLock("lock:report:" + reportId);
    // tryLock: 尝试 0 秒获取, 拿不到立刻返回, 不等待
    if (!lock.tryLock()) {
        return Result.fail("报告正在生成中, 请勿重复提交");
    }
    try {
        // 提交异步任务
        xxlJobHelper.submitTask("reportGenerateJob", reportId);
        return Result.ok("已提交生成");
    } finally {
        // 注意: 不要立刻 unlock! 这里还没生成完。
        // 应该让任务完成后回调 unlock
        // 或者用 SETNX 做提交去重, 不用锁控制执行过程
    }
}
```

更好的方案：用 Redis SETNX 做请求去重，不阻塞；任务状态用 Redis key `report:status:{id}` 标记（PENDING/PROCESSING/DONE/FAILED）。

Q30: 缓存不一致问题——失效模式 vs 双写模式

Cache Aside（旁路缓存）——推荐：

读：先读缓存 → 命中返回 | 未命中 → 查 DB → 写缓存 → 返回
写：先更新 DB → 再删除缓存

为什么是删缓存而不是更新缓存？

- 更新缓存可能浪费（更新后一直没人读）
- 并发下更新顺序问题可能导致脏数据
- 删除 + 下次读时重建是更安全的模式

延时双删：先删缓存 → 更新 DB → 等待（如 200ms）→ 再删一次缓存

- 解决：更新 DB 期间，其他线程读到旧数据写入了缓存

双写模式（不推荐）：同时更新 DB 和缓存 → 操作非原子，数据可能不一致

最终一致性方案：

- Canal 监听 binlog → 异步更新/删除缓存
- MQ 保证消息投递 → 消费端更新缓存

六、消息队列 (MQ)

Q31: RocketMQ 消息堆积怎么办？

原因排查：

1. 消费者处理能力不足（最常见），消费耗时大于生产速度

2. ConsumerGroup 下消费者实例异常、线程池打满或消费线程阻塞
3. 生产端流量突增，例如批量导入、集中考试、第三方回调高峰
4. 消费者代码性能问题，例如慢 SQL、远程接口超时、单条消息处理太重
5. Topic 队列数太少，消费者实例增加后也无法继续提升并发

解决方案（优先级从高到低）：

1. **临时扩容**：增加消费者实例数量
2. **提高消费并发度**：调大消费线程池，前提是数据库和下游扛得住
3. **批量消费**：一次拉取多条消息，攒批写库或批量调用
4. **跳过非关键消息**：降级策略，丢弃部分日志/通知类消息
5. **消费端性能优化**：SQL 优化、批量写库、异步化、缓存
6. **增加队列数或迁移 Topic**：队列数不足时，单纯加消费者没有效果
7. **限制生产者**：上游增加限流

Q32: RocketMQ 重复消费 & 消费成功保证

重复消费原因：

- 生产端：网络超时重试
- Broker：消息投递成功但响应丢失，生产者重试
- 消费端：业务处理成功但返回消费成功前服务挂了，Broker 后续重新投递
- Rebalance：消费者扩缩容或重启时，消费队列重新分配，可能出现重复投递

消费成功保证：

- RocketMQ 消费者返回 CONSUME_SUCCESS 后，Broker 才认为消费成功并推进消费位点
- 返回 RECONSUME_LATER 或抛异常后，Broker 会按重试策略重新投递
- 多次重试仍失败会进入死信队列，需要人工或补偿任务处理
- 不要把“收到消息”当成“业务成功”，必须业务事务提交后再返回成功

幂等消费方案： | 方案 | 实现 | |-----|-----| | 数据库唯一键 + 插入 | 用 msgId 做唯一键，重复消息插入失败 catch 忽略 | | Redis SETNX | SETNX msg:consumed:{msgId} 1，消费前去重 | | 版本号乐观锁 | UPDATE xxx SET status=2 WHERE id=1 AND status=1，判断影响行数 | | 流水表 | 消费前查消费记录表，已存在则跳过 |

Q33: RocketMQ 消费慢、重试多怎么排查？

核心指标：

- 消费堆积量：ConsumerGroup 在 Topic 上的 lag
- 消费耗时：单条消息平均处理时间和 P95/P99
- 重试次数：是否频繁进入 retry topic
- 死信消息：是否进入 %DLQ%{consumerGroup}

排查路径：

```

发现消息堆积
|
v
看 ConsumerGroup 是否在线
|
v
看消费线程是否阻塞 / 线程池是否打满
|

```

v
看业务耗时：SQL / Redis / 第三方接口
|
v
看失败消息是否反复重试
|
v
扩容消费者 / 优化业务 / 限流生产者 / 处理死信

口述答案：

RocketMQ 里我会先看 ConsumerGroup 的消费堆积和消费者是否在线。如果消费者在线但堆积持续增加，通常是消费速度跟不上生产速度，需要看单条消息处理耗时、线程池、SQL 和下游接口。如果失败消息反复重试，要先修复业务异常或把异常消息转人工处理，否则会一直拖慢正常消息。

补充：国内 Java 项目里 RocketMQ、RabbitMQ、Kafka 怎么选？

结论：

MQ	更常见场景	面试表达
RocketMQ	国内 Java 微服务、订单/交易/业务异步、延迟消息、事务消息	简历主线推荐
RabbitMQ	传统业务系统、轻量异步、路由灵活、历史项目较多	可以了解和对比
Kafka	日志、埋点、大数据、流式处理、高吞吐	不要包装成业务事务 MQ

口述答案：

国内 Java 微服务项目里，如果技术栈是 Spring Cloud Alibaba、Nacos、Sentinel 这一套，RocketMQ 更贴近常见选型。它在业务异步、削峰、延迟消息、消费重试、死信和事务消息方面都比较常见。RabbitMQ 也有不少项目使用，尤其是传统系统和轻量异步场景。Kafka 更偏日志、埋点和大数据流处理，不太适合包装成订单状态流转这类强业务消息的首选。

限流操作：

Spring Cloud Gateway + Redis Rate Limiter

```
spring:
  cloud:
    gateway:
      routes:
        - id: buy_route
          uri: lb://order-service
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter:
                  replenishRate: 10    # 每秒10个令牌
                  burstCapacity: 20    # 桶容量20
```

```
@Bean
public KeyResolver userKeyResolver() {
    return exchange -> Mono.just(
        exchange.getRequest().getQueryParams().getFirst("userId")
    );
}
```

```
    );  
}
```

七、MyBatis 相关

Q34: MyBatis 一对多怎么查

主键必须使用 `<id>`, MyBatis 通过 `<id>` 判断是否是同一对象, 否则会把每条关联表记录都当作新对象。

```
<!-- 方式一: collection 嵌套结果映射 (推荐, 一次查询) -->  
<resultMap id="DeptWithUsers" type="Dept">  
    <id property="id" column="dept_id"/>  
    <result property="name" column="dept_name"/>  
    <collection property="users" ofType="User">  
        <id property="id" column="user_id"/>  
        <result property="name" column="user_name"/>  
    </collection>  
</resultMap>  
  
<select id="getDeptWithUsers" resultMap="DeptWithUsers">  
    SELECT d.id AS dept_id, d.name AS dept_name,  
           u.id AS user_id, u.name AS user_name  
    FROM dept d LEFT JOIN user u ON d.id = u.dept_id  
    WHERE d.id = #{id}  
</select>
```

两种方式:

- 嵌套结果 (JOIN): 一次查询, resultMap 处理映射
- 嵌套查询 (子查询): N+1 问题, 不推荐

八、性能优化与高并发

Q35: 接口很慢怎么优化?

排查链路:

1. 定位慢在哪一层
 - └─ 前端 → Network 面板看请求耗时
 - └─ 网关 → 日志看转发耗时
 - └─ 负载均衡 → 是否路由到异常节点
 - └─ 应用层 → Arthas trace 追踪方法耗时
 - └─ 数据库 → Druid 慢 SQL 监控 / EXPLAIN
 - └─ 第三方 → 调用超时设置 / 异步化
2. 针对性优化
 - └─ SQL 慢 → 加索引 / 优化语句 / 加缓存
 - └─ 循环调 DB → 批量查询 / JOIN
 - └─ 串行改并行 → CompletableFuture
 - └─ 数据量大 → 分页 / 游标
 - └─ 远程调用多 → 并行调用 / 减少链路
 - └─ 计算密集 → 异步化 / 预计算

顺着走一遍：

1. **定界（前端/网络/后端）**：先看Network面板，确定是后端响应慢（TTFB长）还是传输慢。
2. **定位（后端哪部分慢）**：看SkyWalking链路，找到耗时最长的那个服务。
3. **定责（代码/数据库/中间件）**：在这个服务上用Arthas的 `t trace`，找到耗时最长的方法。
4. **止血**：
 - **DB问题**：EXPLAIN -> 加索引/覆盖索引/干掉 SELECT * -> 分页优化。
 - **锁问题**：杀长事务，改逻辑顺序。
 - **代码问题**：批量化、异步化、并行化。
 - **架构问题**：上缓存、读写分离、消息队列。
5. **验证**：上线后用SkyWalking对比前后的响应时间，用数据说话，别用“我感觉”

Q36: 接口要求 QPS 200 怎么做

逐步拆解：

- QPS 200 = 每秒 200 个请求，每个请求处理窗口 5ms
- **系统承载力**：如果单接口 50ms，最少需要 10 个并发线程（ $200 \times 0.05 = 10$ ）
- **压测验证**：JMeter/Gatling 压测找瓶颈
- **缓存**：热点数据 Redis 缓存
- **异步**：非核心逻辑消息队列异步处理
- **限流**：Sentinel/Redis 滑动窗口防止超量打垮系统

Q37: POI 导入数据量过大导致 OOM

原因：XSSFWorkbook 全量加载到内存 + 大量 String 对象（每个单元格都是 String 对象）

解决：

- **SAX 模式（XSSF + SAX）**：基于事件驱动的流式读取，逐行解析不占内存
- **EasyExcel**：阿里开源，底层 SAX 模式，使用 listener 逐行处理
- **分批提交**：每 1000 行 insert 一次，用完 gc
- **异步化**：上传文件到 OSS → 后台任务处理 → 通知结果

POI有两种模式：

模式	代表类	内存占用	适用场景
UserModel（用户模型）	XSSFWorkbook	全部加载到内存	小文件（<10MB）
EventModel（事件模型）	XSSFWorkbook + SAX	几乎不占内存	大文件（>10MB）

Q38: 100w 用户积分每年 1.1 清零（场景题）

分析：100w 用户 × 一次 update 全部清零 = 锁表/主从延迟/耗时极长。

方案：

1. **不物理清零**：加年度字段 `year_2025_points`，查询时只查当前年；缺点每年增加一个字段，或者使用一个json。
2. **如果必须清零**：分批更新，每批 5000 条，`WHERE id BETWEEN ? AND ?`，任务间隔 100ms，持续数小时完成
3. **XXL-Job 定时**：1月1日凌晨 2:00 启动分片任务
4. **Redis 缓存年度积分**：清零前预存 Redis，防止瞬时流量打到 DB
5. 提前几天清理不活跃的用户积分

九、场景设计 & 杂项

Q39: 接口调用失败的重试机制

```
// Spring Retry
@Retryable(maxAttempts = 3, backoff = @Backoff(delay = 1000, multiplier = 2))
public Result callExternalApi() { }

// 自定义：指数退避 + 熔断器
int retry = 0;
while (retry < 3) {
    try { return doCall(); }
    catch (Exception e) {
        retry++;
        if (retry == 3) throw e;
        Thread.sleep(1000 * (long) Math.pow(2, retry)); // 2s, 4s
    }
}
```

关键点：必须是幂等操作才能重试；非幂等（如扣款）需要查询确认后重试。

Q40: 接口安全验证

- 认证：JWT Token / OAuth2 / Session
 - 防篡改：请求参数签名（MD5/SHA）+ 时间戳防重放
 - 防伪造：HTTPS + 密钥 + 非对称加密
 - 限流：IP + 用户多维度限制
 - 敏感数据：脱敏（身份证/手机号中间打*）
 - SQL 注入：参数化查询（MyBatis 用 `#{}` 不用 `${}` ）
 - XSS：输出编码/过滤
-

Q41: Arthas trace 排查接口慢

```
# 安装
curl -O https://arthas.aliyun.com/arthas-boot.jar
java -jar arthas-boot.jar

# 追踪方法调用链及每个节点耗时
trace com.xxx.controller.ReportController generateReport -n 5

# 监控方法出入参和返回值
watch com.xxx.service.ReportService generate "{params,returnObj,throwExp}"

# 查看方法调用路径
stack com.xxx.service.ReportService generate
```

Q42: Linux 前端调不到后端 Controller

排查链路（从前到后）：

1. 前端 F12 → Network → 确认请求发出, 看状态码
 - └─ 404: URL 错误或没注册
 - └─ 500: 后端报错, 看日志
 - └─ CORS 错误: 跨域没配
2. 后端日志 → `tail -f app.log | grep "请求路径"`
3. `netstat -tlnp | grep 8080` → 确认端口监听
4. `curl localhost:8080/api/xxx` → 本机测试
5. `telnet <ip> 8080` → 检查网络连通性
6. `iptables -L -n` → 检查防火墙规则
7. nginx 层 → `cat /var/log/nginx/error.log`

Q43: WPS 在线编辑接口

一般指 WPS 开放平台的 WebOffice SDK:

- 前端加载 WPS WebOffice 组件, 传入 fileId
- 后端提供三个回调接口: 获取文件信息、保存文件、获取用户信息
- 文件存储在 OSS/COS 上, 通过签名 URL 访问
- 多人协同需要配合 WebSocket 推送编辑状态

Q44: Activity/Flowable workflow 核心表

分类	表名	说明
流程定义	act_re_procdef	流程定义
运行实例	act_ru_execution	执行实例
运行实例	act_ru_task	当前任务/节点
历史	act_hi_procinst	历史流程实例
历史	act_hi_taskinst	历史任务实例
历史	act_hi_actinst	历史活动实例
身份	act_id_user	用户
身份	act_id_group	用户组

Q45: 为什么 Spring Boot 的 jar 可以是 Web 应用

普通 jar 没有 Servlet 容器。Spring Boot jar 里嵌入了 Tomcat (spring-boot-starter-web 中 tomcat-embed-core), JarLauncher 启动时创建内嵌 Tomcat 实例、注册 DispatcherServlet 并监听端口。

jar 包结构:

```
BOOT-INF/
  lib/      → 依赖 jar (含 tomcat-embed)
  classes/  → 应用代码
META-INF/
```

十、快速索引

关键词	对应问题
HashMap/扩容	Q1
String/常量池	Q2
int vs Integer	Q3
run vs start	Q4
单例/DCL	Q5
去重	Q6
list转map	Q7
线程安全	Q8
线程池	Q9
死锁	Q10
Bean 线程安全	Q12
@Transactional	Q13
Spring 事务	Q14
Spring MVC vs Boot	Q15
@SpringBootApplication	Q16
Spring Security	Q17
Feign	Q18
微服务体系	Q19
B+ 树	Q20
回表	Q21
最左匹配/索引失效	Q22
INT(10) vs INT(11)	Q23
ORDER BY 失效	Q24
大表查询优化	Q25
千万级数据	Q26
分库分表	Q27
Redisson/看门狗	Q28
报告生成场景	Q29
缓存不一致	Q30
消息堆积	Q31
重复消费/幂等	Q32
RocketMQ 消费慢/重试	Q33
MQ 选型	补充: RocketMQ/RabbitMQ/Kafka
MyBatis 一对多	Q34
接口慢排查	Q35
QPS 200	Q36
POI OOM	Q37
积分清零	Q38

关键词	对应问题
重试机制	Q39
接口安全	Q40
Arthas trace	Q41
Linux 排查	Q42
WPS 在线编辑	Q43
Activity 表	Q44
Spring Boot jar	Q45
分布式调度	Q46
分布式事务	Q47
Prototype 注入	Q48
CompletableFuture	Q49
MongoTemplate/MongoRepository	Q50
项目介绍	Q51 / Q68
网关 JWT	Q52
电子发票幂等	Q53 / Q65
第三方接口设计	Q54
视频转码异步	Q55
CNRDS 上报	Q56
库存并发扣减	Q57
XXL-Job 分片	Q58
慢 SQL 排查	Q59
多租户隔离	Q60 / Q61
RocketMQ 可靠消费	Q62
独立负责怎么说	Q63
医院检查结果同步设计	Q64
线上接口变慢排查	Q66
苏州 3-5 年追问准备	Q67
JVM 生命周期/类加载/内存	Q69 / Q70 / Q71
朋友面经真题对照	十五
Nacos vs Eureka/Apollo	Q72
Sentinel vs Hystrix	Q73
MyBatis-Plus 多租户	Q74
深度分页	Q75
MyBatis-Plus vs JPA	Q76
雪花算法 ID 重复	Q77
大文件分片上传 Java	Q78
工厂/策略模式 Java	Q79
IOC/DI、RabbitMQ unack、连接池	十五 补充 1~4
Java 项目讲稿（口述）	十七

十一：补充

Q46: 分布式调度

为什么需要分布式调度？

单机定时任务（如 `@Scheduled`）在多节点部署时会产生以下问题：

- **重复执行**：多台机器同时触发同一任务（如每天凌晨结算，每个节点都跑一遍）
- **无法动态扩缩**：不能灵活分配任务到不同节点
- **无失败重试/告警**：任务挂了没人知道，缺少运维能力
- **无分片能力**：大数据量任务无法拆分并行处理

主流方案对比：

特性	Quartz	XXL-Job	Elastic-Job
分布式协调	JDBC 行锁	中心化调度中心	ZooKeeper
分片	不支持	支持（广播+分片参数）	原生支持
管理界面	无（需自建）	自带 Web 控制台	自带控制台
动态添加任务	不支持	支持（控制台操作）	支持
失败重试	需自实现	内置	内置
社区活跃度	一般	极高	停更（已捐赠 Apache）

XXL-Job 核心原理：

调度中心 (Admin)

- ├ 管理任务、触发调度、监控日志
- └ 通过 HTTP 回调执行器

执行器 (Executor, 嵌入业务服务)

- ├ 启动时注册到调度中心
- ├ 接收调度请求，执行任务
- └ 30s 一次心跳，90s 无心跳剔除

分片广播机制：

当任务设置为"广播+分片"模式时，调度中心将所有执行器实例的路由策略置为"分片广播"，每个执行器收到参数 `shardIndex`（当前分片序号）和 `shardTotal`（总分片数）。

```
// 3 台机器，处理 300 万条数据
// 执行器 0: id % 3 = 0 的数据 → 100万条
// 执行器 1: id % 3 = 1 的数据 → 100万条
// 执行器 2: id % 3 = 2 的数据 → 100万条
@XxlJob("cleanExpiredData")
public void cleanExpiredData() {
    int shardIndex = XxlJobHelper.getShardIndex(); // 0/1/2
    int shardTotal = XxlJobHelper.getShardTotal(); // 3
    List<Long> ids = mapper.getIdsByShard(shardIndex, shardTotal);
    // 逐批处理...
}
```

实际场景（积分清零）： 100w 用户积分每年 1.1 清零 → XXL-Job 凌晨 2:00 触发，3 节点分片，每节点处理 ~33w 用户，单节点内 5000 条一批，分页更新。

Q47: 分布式事务

当前简历口径：可以把“分布式事务”作为高级技术栈准备，但项目中主讲可靠消息、状态机、补偿任务这类最终一致性方案；Seata AT/TCC 作为可对比方案了解适用边界，不要把第三方 HTTP 接口包装成 Seata 强事务。

产生背景：

单体架构 → 微服务拆分后，一个“下单”操作可能跨 3 个服务（订单服务、库存服务、优惠券服务），每个服务有独立的数据库。单机 @Transactional 只能管自己的库，需要分布式事务保证跨服务的数据一致性。

理论基础：

CAP 定理：分布式系统无法同时满足 C（一致性）、A（可用性）、P（分区容错性），P 必须保证，只能在 C 和 A 之间权衡。

BASE 理论：

基本可用（Basically Available）：允许部分功能降级

软状态（Soft State）：允许系统存在中间状态

最终一致性（Eventually Consistent）：数据最终一致即可

方案一：2PC（两阶段提交）

阶段1（Prepare）：协调者问所有参与者——能提交吗？

- └─ 参与者执行 SQL，写 undo/redo 日志，但不 commit
- └─ 回复 YES（可以提交）或 NO（失败）

阶段2（Commit/Rollback）：

- └─ 全部 YES → 协调者发 Commit → 所有参与者提交
- └─ 任一 NO → 协调者发 Rollback → 所有参与者回滚

2PC 致命缺陷：

- **同步阻塞**：prepare 阶段锁资源一直不释放，时间窗口长
- **单点故障**：协调者挂了，所有参与者卡住（参与者有超时机制但协调者没有）
- **数据不一致**：commit 阶段部分节点收到、部分没收到（网络分区）
- **实际很少使用**，只适用于数据库层面（XA 协议）

方案二：3PC（三阶段提交）

2PC 的改良版，在 Prepare 和 Commit 之间插入 PreCommit 阶段，同时给参与者也加了超时机制：

阶段1（CanCommit）：能提交吗？（只询问，不做任何操作）

阶段2（PreCommit）：预执行（写日志但不提交）

阶段3（DoCommit）：真正提交/回滚

改进：

- 参与者在 PreCommit 后超时未收到 DoCommit → **自动提交**（避免了 2PC 的无限等待）
- 代价：可能产生数据不一致（参与者超时后自动提交了，但协调者决定回滚）

实际也很少使用，2PC/3PC 都是刚性事务，性能差。

方案三：TCC（Try-Confirm-Cancel）

核心思想：把业务逻辑拆成三个方法，由业务方自己实现。

Try (预留资源) → 冻结库存 100 件 (还没扣)
Confirm (确认) → 真正扣减库存 100 件
Cancel (回滚) → 解冻库存 100 件

流程：

1. TM 调所有参与者的 Try 方法
2. 全部 Try 成功 → TM 调所有 Confirm
3. 任一 Try 失败 → TM 调所有 Cancel

TCC 三大难题及解法：

问题	场景	解法
空回滚	Try 超时没到 → TM 触发 Cancel → 后来 Try 才到达	Cancel 执行前检查是否 Try 过，没 Try 过直接返回成功
悬挂	Cancel 先到 → Try 后到	Cancel 记录回滚标记，Try 先查标记，已回滚则不执行
幂等	Confirm/Cancel 被重复调	用 xid + branchId 做唯一键，防重复执行

示例 (Seata TCC)：

```
@TwoPhaseBusinessAction(name = "deductStock", commitMethod = "confirm", rollbackMethod = "cancel")
public boolean tryDeduct(@BusinessActionContextParameter(paramName = "skuId") Long skuId,
                        @BusinessActionContextParameter(paramName = "count") Integer count) {
    // Try: 冻结库存
    return stockService.freeze(skuId, count);
}

public boolean confirm(BusinessActionContext ctx) {
    // Confirm: 真正扣减
    Long skuId = (Long) ctx.getActionContext("skuId");
    Integer count = (Integer) ctx.getActionContext("count");
    return stockService.deduct(skuId, count);
}

public boolean cancel(BusinessActionContext ctx) {
    // Cancel: 解冻
    Long skuId = (Long) ctx.getActionContext("skuId");
    Integer count = (Integer) ctx.getActionContext("count");
    return stockService.unfreeze(skuId, count);
}
```

优缺点：

- 优点：性能高（Try 阶段只锁业务资源，不锁数据库行），对数据库无侵入
- 缺点：业务侵入性强，每个服务需要写三个方法，设计复杂

方案四：事务消息（最终一致性）

核心思路：不追求强一致，通过消息队列保证"最终"一致。

RocketMQ 事务消息流程：

1. 生产者发 Half 消息（半消息，对消费者不可见）
2. 生产者执行本地事务
 - └ 成功 → Commit → 消息对消费者可见 → 消费者消费
 - └ 失败 → Rollback → 消息删除
3. 生产者宕机/超时 → MQ 回调 CheckListener 查询本地事务状态
 - └ 已提交 → Commit
 - └ 已回滚 → Rollback

场景示例（下单扣库存）：

1. 订单服务发 Half 消息 "订单创建成功"
2. 订单服务执行本地事务：插入订单表，状态=待支付
3. 成功 → Commit 消息
4. 库存服务消费消息 → 扣库存
5. 如果库存不足 → 发退款补偿消息 → 订单服务标记订单已取消

优缺点：

- 优点：最终一致性，系统解耦，吞吐量高
- 缺点：中间状态不可见（用户看到"处理中"），需要补偿机制

四种方案对比总结：

方案	一致性	性能	侵入性	适用场景
2PC	强一致	极差	低	几乎不用
3PC	强一致	差	低	几乎不用
TCC	最终一致	高	高	核心系统、对性能要求高的场景
事务消息	最终一致	高	中	异步解耦场景（下单→扣库存→发券）
Seata AT	最终一致	中	极低	不想改业务代码，需要快速接入

Q48: Prototype Bean 注入到 Singleton 中为什么会退化？

问题本质：

```

@Component
@Scope("prototype")
public class PrototypeBean { }

@Component
public class SingletonBean {
    @Autowired
    private PrototypeBean prototypeBean; // 只注入一次！
}

```

Spring 的 Singleton Bean 在容器启动时就创建好了，它的依赖也会在那一刻注入。换句话说，SingletonBean 里的 prototypeBean 在整个 Spring 生命周期里永远是同一个对象——第一次注入后不会再换了。

为什么用 @Lookup 能解决？

```

@Component
public class SingletonBean {
    @Lookup

```

```

    public PrototypeBean getPrototypeBean() {
        return null; // Spring 用 CGLIB 重写这个方法
    }
}

```

Spring 通过 CGLIB 动态代理生成 SingletonBean 的子类，重写 @Lookup 注解的方法，每次调用都走 BeanFactory.getBean() 重新获取新的 Prototype 实例。

其他解法：

方式	说明
@Lookup	Spring 原生支持，推荐
ApplicationContext.getBean()	直接拿容器，但耦合 Spring 框架
ObjectFactory / Provider	@Autowired private ObjectFactory<PrototypeBean> factory; 然后 factory.getObject()
把"会变的状态"抽出去	换个思路——不要依赖每次 new 对象，用 ThreadLocal 或 Redis 存状态，Bean 保持无状态

核心结论：在 Spring 容器里，不要期望注入的 Prototype Bean 会"自动刷新"，Spring 只负责注入那一次。需要"每次用新的"，就要通过 @Lookup、ObjectFactory 或 Provider 主动获取。

Q49: CompletableFuture 异常处理

内容如上文代码示例，这里补充核心结论

allOf().join() 的坑：

- allOf 返回的 CompletableFuture 只要所有任务**结束**（不管成功还是异常）就算完成
- join() 不会因为某个子任务异常而抛异常
- 子任务的异常会被**静默吞掉**

三种处理方式对比：

方式	适用场景	缺点
每个 future 内部 try-catch + 收集异常	需要精确知道哪个任务失败	代码冗余
handle() 统一收尾	每个任务都有明确的成功/失败结果	返回值类型统一
exceptionally() 只处理异常	只需要降级兜底	正常逻辑和异常逻辑分开
orTimeout() + completeOnTimeout()	超时控制	Java 9+

```

// Java 9 超时控制
CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> callRemoteService())
    .orTimeout(3, TimeUnit.SECONDS)
    .completeOnTimeout("降级默认值", 3, TimeUnit.SECONDS);

```

Q50: Spring Boot 操作 MongoDB——MongoTemplate

在生产项目中，最推荐的方式是 **MongoTemplate**。MongoRepository 只适合简单 CRUD，遇到条件查询、聚合、索引管理就力不从心了。实际项目通常两者共存，但核心操作走 MongoTemplate。

依赖：

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>

```

配置：

```

spring:
  data:
    mongodb:
      uri: mongodb://localhost:27017/invoice_db
      # 或分开配置
      # host: localhost
      # port: 27017
      # database: invoice_db

```

实体映射：

```

@Document(collection = "invoice_log") // 映射到集合
public class InvoiceLog {
    @Id
    private String id; // MongoDB 自动生成的 _id
    @Field("tenant_id")
    private String tenantId;
    private String requestBody;
    private String responseBody;
    @Field("create_time")
    private Date createTime;
    @Indexed // 单字段索引
    private String serialNo;
}

```

条件查询 (Query + Criteria) :

```

@Autowired
private MongoTemplate mongoTemplate;

// 精确匹配 + 范围 + 排序 + 分页
public List<InvoiceLog> queryLogs(String tenantId, Date start, Date end, int page, int
size) {
    Query query = new Query(
        Criteria.where("tenant_id").is(tenantId)
            .and("status").in("SUCCESS", "FAILED")
            .and("create_time").gte(start).lte(end)
    );
    query.with(Sort.by(Sort.Direction.DISC, "create_time"));
    query.skip((long) (page - 1) * size).limit(size);
    return mongoTemplate.find(query, InvoiceLog.class);
}

```

批量更新：

```

// 7 天前的临时数据标记为过期
Update update = new Update().set("status", "EXPIRED");
mongoTemplate.updateMulti(
    Query.query(Criteria.where("create_time").lt(sevenDaysAgo)),

```

```
        update,  
        InvoiceLog.class  
    );
```

聚合管道（分组统计）：

```
// 按渠道统计开票量  
Aggregation agg = Aggregation.newAggregation(  
    Aggregation.match(Criteria.where("status").is("SUCCESS")),  
    Aggregation.group("channel").count().as("total"),  
    Aggregation.sort(Sort.Direction.DESC, "total")  
);  
AggregationResults<ChannelStat> results = mongoTemplate.aggregate(  
    agg, "invoice_log", ChannelStat.class  
);
```

TTL 索引——自动过期删除：

这是 MongoTemplate 最实用的特性之一，比 Redis TTL 更适合大体积数据的自动清理：

```
// 创建 TTL 索引：create_time 7 天后文档自动删除  
mongoTemplate.indexOps("invoice_cache")  
    .ensureIndex(new Index().on("create_time", Sort.Direction.ASC)  
        .expire(7, TimeUnit.DAYS));
```

为什么不用 MongoRepository？

MongoRepository 只适合 `findByName`、`findById` 这类简单场景。一旦涉及多条件组合查询、聚合统计、索引管理，它就搞不定了。实际项目中通常 Service 层同时注入 MongoTemplate 和 MongoRepository，简单读写走 Repository，复杂操作走 Template，各取所长。

十二、结合简历项目的面试题

这一部分建议重点背熟。回答时不要说“我主导了整体架构”，更稳妥的说法是：

我负责其中的接口设计、核心代码开发、联调和上线问题处理；整体架构是团队统一方案，我主要负责把这个模块落地，并处理过程中遇到的幂等、缓存、异步、数据一致性和性能问题。

Q51: 你在项目里做过哪些比较有代表性的 Java 后端工作？

回答结构：

1. 先说业务：医疗 HDIS、CNRDS 上报、电子发票网关、专家培训考试。
2. 再说职责：负责模块开发、表结构设计、接口设计、联调、上线排查。
3. 最后说技术点：Redis 幂等/限流、MQ 异步、XXL-Job 分片、SQL 优化、第三方接口签名。

可直接口述：

我主要做过两类项目。一类是医疗信息化项目，比如血液透析系统，涉及患者建档、透析排班、处方、结算、库存和 CNRDS 上报；另一类是政企服务平台，比如专家培训考试、电子发票网关和金融保函。我的职责不是整体架构决策，更多是负责具体模块或子系统的后端落地，包括接口设计、表结构设计、核心代码开发、前后端联调、上线后的问题排查。技术上用得比较多的是 Spring Boot、Spring Cloud Alibaba、MyBatis-Plus、Redis、RocketMQ、XXL-Job、MongoDB 和 MySQL。

Q52: 你们网关 JWT 认证是怎么做的？

回答流程：

```
graph TD
    A[用户登录] --> B[认证接口校验账号密码]
    B --> C[生成 Access Token + Refresh Token]
    C --> D[Refresh Token / 黑名单写入 Redis]
    D --> E[后续请求携带 Access Token]
    E --> F[Gateway GlobalFilter 校验 Token]
    F --> G[解析 userId / tenantId / role 写入请求头]
    G --> H[转发到后端业务服务]
```

口述答案：

我们是通过 Gateway 做统一入口。登录接口校验账号密码后，会生成 Access Token 和 Refresh Token。Access Token 有较短有效期，Refresh Token 放 Redis，方便续签和主动下线。后续请求进入网关后，GlobalFilter 会解析 Token，校验签名、过期时间和 Redis 黑名单。如果通过，就把 userId、tenantId、role 等上下文写到请求头里，后端服务只需要从请求头里取当前用户信息。

追问：Token 主动失效怎么做？

用户退出或管理员踢人时，把 Token 的 jti 或 token 字符串写入 Redis 黑名单，TTL 设置为 Token 剩余有效期。网关每次校验时先查黑名单，命中就拒绝。

追问：为什么不每个服务都解析 Token？

统一放网关处理可以减少重复代码，也能统一跨域、日志、IP 黑名单、限流等公共能力。后端服务还是要做方法级权限校验，网关负责认证和基础拦截，业务服务负责细粒度授权。

Q53: 电子发票网关如何防止重复开票？

核心思路：

- 业务流水号(bizid)必须唯一。
- 请求进入先做幂等校验。
- 外部接口超时不能盲目重试开票，要先查状态。
- 成功/失败/处理中/部分开票/红冲状态要落库，方便补偿。

流程图：



口述答案：

电子发票这类接口最怕重复开票，所以我会先保证订单号、流水号和明细状态唯一。请求进来后用 Redis SETNX 按流水号加幂等标记，如果加锁失败，说明同一笔业务正在处理或已经处理过，就查询本地开票记录返回已有状态。真正调用第三方前会先落一条开票记录，状态是处理中；第三方返回后再更新成功、失败、部分开票或红冲状态。如果调用超时，不会直接重复开票，而是优先查第三方订单状态或者走人工/补偿流程。

追问：Redis 锁释放前服务挂了怎么办？

SETNX 必须带过期时间，避免死锁。本地库还要有唯一索引兜底，因为 Redis 只能降低重复请求概率，真正的数据库一致性要靠数据库唯一约束和业务状态机保证。

Q54: 第三方接口对接你一般怎么设计？

回答框架：



口述答案：

我一般不会把第三方接口逻辑直接写在业务 Service 里，而是抽一层适配器。内部定义统一的申请、查询、回调处理接口，不同厂商实现自己的报文组装、签名、加密、验签和状态码转换。业务层只关心统一入参和统一结果。这样后面新增厂商时，主要新增一个 Adapter 实现类，不需要大量改原来的业务流程。

追问：第三方接口不稳定怎么办？

我会从四个方面处理：超时时间设置合理；失败重试只针对幂等查询类接口；核心写操作先落库再异步补偿；所有请求和响应都保存日志，方便排查和人工补偿。

Q55: 视频转码为什么要异步？你们流程怎么做？

为什么异步：

视频转码耗时长，如果上传接口同步等待 FFmpeg 完成，HTTP 请求会超时，也会占用 Tomcat 线程，影响其他用户操作。

流程图：

```
graph TD
    A[前端上传视频到七牛云] --> B[后端保存文件信息(status=UPLOADED / TRANSCODING)]
    B --> C[进入独立转码服务队列]
    C --> D[转码服务执行 FFmpeg]
    D --> E["+-> 转码成功：更新 status=SUCCESS，保存播放地址"]
    D --> F["+-> 转码失败：更新 status=FAILED，记录失败原因"]
    E --> G[回调接口更新状态，前端查询状态]
    F --> G
```

口述答案：

视频上传和转码我会拆开处理。视频先上传到七牛云，业务系统保存文件信息和转码状态，转码服务从队列里取任务调用 FFmpeg 生成不同清晰度的视频。转码完成后通过回调接口通知业务系统更新状态，未完成时前端显示“转码中”。WebSocket 主要用于学习过程中的人机验证弹窗和学习进度提醒，不作为转码完成通知的主通道。

追问：多节点下怎么避免同一个视频被多个节点转码？

可以用任务状态 + 乐观锁，或者 Redis 分布式锁。比如任务表里 status 从 WAITING 更新为 PROCESSING 时带上原状态条件，只有更新成功的节点才能执行。Redis 锁也要设置过期时间，避免节点挂掉后任务永远卡住。

Q56: CNRDS 数据上报你具体做了什么？

回答重点：

- 数据来源多：患者、诊断、治疗、用药、检验检查、透析记录。
- 难点不是单纯调用接口，而是字段映射、编码转换、校验、补报。
- 上报需要可追踪：批次、状态、失败原因、重试次数。

流程图：



口述答案：

CNRDS 上报不是简单调接口，主要难点在数据质量。我要从患者基本信息、治疗记录、实验室检查、用药、诊断等业务表里抽取数据，再做字段映射和编码转换。上报前会做必填、范围、字典值和关联关系校验，校验不通过的数据不会直接上报，而是记录失败原因给业务人员修正。真正上报时按批次记录日志，保存每条数据的状态、失败原因和重试次数，后续可以手动补报或定时重试。

追问：如果国家平台临时不可用怎么办？

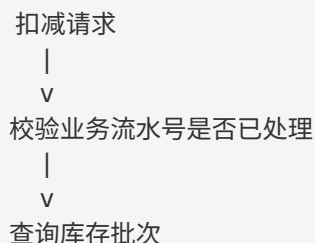
先把本地待上报数据落库，状态标记为待上报或失败，不阻塞医院正常业务。通过定时任务重试，重试次数达到上限后告警或转人工处理。

Q57: 库存扣减如何保证并发安全？

常见问题：

- 两个人同时扣同一批次库存，可能超卖。
- 请求重复提交，可能重复扣减。
- 扣减失败后需要可追踪、可回滚。

推荐设计：




```

|
v
UPDATE stock
  SET qty = qty - ?,
      version = version + 1
WHERE id = ?
  AND qty >= ?
  AND version = ?
|
+--> affected rows = 1: 扣减成功, 写流水
|
+--> affected rows = 0: 库存不足或并发冲突, 重试/返回失败

```

口述答案:

库存扣减我会用数据库条件更新和乐观锁兜底。更新 SQL 里带上库存数量条件和 version 条件，只有库存足够并且版本号没变才更新成功。扣减成功后写库存流水，记录业务单号、批号、数量和操作人。接口层再用业务流水号做幂等，防止前端重复点击或者接口重试导致重复扣减。

追问：为什么不用 synchronized?

单机 synchronized 在多节点部署下无效，而且锁在 JVM 内存里，不能保护数据库层面的并发。库存最终一致性要靠数据库条件更新、唯一约束、事务和流水记录保证。

Q58: XXL-Job 分片任务你怎么用?

适用场景:

- CNRDS 批量补报
- HIS 数据同步
- 大批量历史数据修复
- 年度积分清零或状态刷新

分片流程:

```

调度中心触发任务
|
v
3 个执行器同时收到任务
|
+--> shardIndex=0, shardTotal=3, 处理 id % 3 = 0
|
+--> shardIndex=1, shardTotal=3, 处理 id % 3 = 1
|
+--> shardIndex=2, shardTotal=3, 处理 id % 3 = 2

```

口述答案:

XXL-Job 分片适合处理大量数据。比如有 100 万条待同步数据，3 个执行器实例可以按 id 取模拆分，每个实例只处理自己分片的数据。单个实例内部再分页处理，比如每批 500 或 1000 条，避免一次性加载太多数据导致内存压力。每批处理后记录成功数、失败数和最后处理位置，失败数据单独记录，方便后续补偿。

追问：任务执行一半失败怎么办?

任务要设计成可重入。每条数据有状态字段，比如 WAITING、PROCESSING、SUCCESS、FAILED。重跑时只捞 WAITING 或 FAILED 的数据，已经 SUCCESS 的不再处理。必要时 PROCESSING 超过一定时间也可以重置为 WAITING。

Q59: 慢 SQL 你一般怎么排查？

排查路径：

```
发现接口慢
|
v
看接口日志：controller/service/mapper 耗时
|
v
定位具体 SQL
|
v
EXPLAIN 查看执行计划
|
+--> type 是否太差
+--> key 是否命中索引
+--> rows 扫描行数是否过大
+--> Extra 是否 filesort / temporary
|
v
调整索引 / SQL / 分页 / 缓存
|
v
压测或对比优化前后耗时
```

口述答案：

我一般先看接口分段耗时，确认慢在业务逻辑、远程调用还是数据库。如果定位到 SQL，就用 EXPLAIN 看有没有命中索引、扫描行数、是否 Using filesort 或 Using temporary。优化方式包括补联合索引、调整最左匹配顺序、减少 select *、减少无意义 join、分页改成基于游标或 lastId、统计类接口加缓存。优化后要对比执行计划和实际耗时，不能只凭感觉。

项目化例子：

透析记录和费用报表查询里，常见条件是 tenant_id、patient_id、treat_date、status。我会优先考虑按租户 + 时间 + 患者这类高频过滤条件建立联合索引，同时避免在索引字段上使用函数，减少排序和回表。

Q60: 多租户数据隔离怎么做？有什么风险？

基本方案：

```
请求进入网关
|
v
解析 Token 得到 tenantId
|
v
tenantId 写入请求头
|
v
```

```
后端拦截器/上下文保存 tenantId
|
v
MyBatis 查询自动追加 tenant_id 条件
|
v
返回当前租户数据
```

口述答案：

我们这种系统更多是共享库共享表，通过 tenant_id 做逻辑隔离。用户登录后，网关解析 Token，把 tenantId 透传给后端。后端查询时通过拦截器或统一查询条件自动追加 tenant_id，业务代码也会在新增数据时写入 tenant_id。这样可以减少每个接口手写租户条件的遗漏。

风险点：

- 手写 SQL 或复杂报表 SQL 容易漏 tenant_id。
- 管理后台跨租户查询必须做明确权限控制。
- 缓存 Key 必须带 tenantId，否则可能串数据。
- 异步任务和 MQ 消息也要携带 tenantId。

Q61: Redis 缓存如何避免多租户串数据？

错误示例：

```
cache:user:1001
```

如果多个租户都有 userId=1001，就会互相覆盖。

正确示例：

```
cache:tenant:{tenantId}:user:{userId}
cache:tenant:{tenantId}:dict:{dictType}
cache:tenant:{tenantId}:report:{date}
```

口述答案：

多租户系统里缓存 Key 必须带 tenantId，尤其是用户、字典、报表、权限这类数据。否则不同院区或不同租户出现相同业务 ID 时会串数据。除了缓存 Key，MQ 消息、异步任务参数、导出文件名和临时数据也要带租户标识。

Q62: RocketMQ 消息如何保证不丢、不重复影响业务？

回答要点：

- 不丢：Broker 持久化、生产发送结果确认、消费失败重试、死信队列。
- 不重复影响：消费端幂等。
- 不能只依赖 MQ 保证“只消费一次”。

流程图：

```
生产者发送消息
|
v
```

```
发送结果确认到达 Broker
|
v
Topic/MessageQueue 持久化
|
v
消费者拉取消息
|
v
业务处理 + 幂等校验
|
+--> 成功: 返回 CONSUME_SUCCESS
|
+--> 失败: RECONSUME_LATER / retry / dead letter / 记录失败
```

口述答案:

RocketMQ 可靠性我会分生产端、Broker 和消费端来看。生产端要判断发送结果，失败时按业务场景重试或落本地异常记录；Broker 负责消息持久化；消费端只有在业务事务真正提交后才返回 `CONSUME_SUCCESS`。如果消费失败，返回 `RECONSUME_LATER`，RocketMQ 会重新投递，多次失败后进入死信队列。重复消费不可避免，所以消费端必须做幂等，比如用业务单号建唯一索引，或者用 Redis `SETNX` 判断是否处理过。在河南产权里，我会把它讲成异步通知、转码、回调和状态流转的可靠消息，而不是只说“用了个 MQ”。

Q63: 面试官问“你独立负责过什么”怎么回答更可信？

推荐回答:

我独立负责过一些边界比较清晰的子系统或模块，比如 CMS 存量资产系统、电子发票网关里的开票记录和第三方适配模块、HDIS 里的 CNRDS 上报和库存部分功能。我的独立主要是指能自己完成需求拆解、接口设计、表设计、编码、联调和上线问题处理，不是说整个项目的架构和技术选型都是我一个人决定的。

不要这样说:

- 我主导了整个微服务架构。
- 我负责公司核心平台从 0 到 1。
- 我设计了整个 SaaS 多租户架构。

可以这样说:

- 我参与了整体方案讨论，负责其中某个模块落地。
- 这个子系统边界比较清晰，我能独立推进。
- 架构方案是团队统一的，我主要负责实现和问题处理。

Q64: 如果让你设计一个医院检查结果同步子系统，你怎么做？

场景:

HIS/LIS 每天产生大量检验检查结果，需要同步到 HDIS，供医生查看和 CNRDS 上报使用。医院现场对接方式可能是 API、WebService 或查询视图，不同医院按实际情况选一种。

设计图:

```
外部 HIS/LIS
|
| HTTP / DB View / 定时文件
```

```

v
数据接入层
|
v
原始数据表 raw_exam_result
|
v
清洗转换任务
|
+--> 单位转换
+--> 编码映射
+--> 必填校验
+--> 去重
|
v
业务结果表 exam_result
|
v
HDIS 页面查询 / CNRDS 上报

```

回答步骤：

1. 先确定对接方式：接口、数据库视图、文件还是消息。
2. 原始数据先落 raw 表，方便追溯。
3. 清洗层做字段映射、单位转换、编码转换、必填校验。
4. 通过外部流水号 + 患者 ID + 检查时间做去重。
5. 使用 XXL-Job 分片定时同步，失败记录原因，支持补偿；如果医院接口是视图拉取，就按医院编码和时间窗口分片处理。
6. 查询侧按 patient_id、exam_time、item_code 建索引。

面试亮点：

这个设计既考虑了业务可追溯，也考虑了失败补偿。医疗数据不能只追求“同步成功”，更要能解释为什么失败、怎么修复、是否可以补报。

Q65: 如果让你设计电子发票回调处理，怎么保证状态正确？

状态机：

```

INIT
|
v
PROCESSING
|
+--> SUCCESS
|
+--> FAILED
|
+--> UNKNOWN --定时查询--> SUCCESS / FAILED

```

关键控制：

- 回调必须验签。
- 回调可能重复，要幂等。
- 状态只能按规则流转，不能从 SUCCESS 改回 FAILED。
- 超时未回调要主动查询第三方状态。

口述答案：

我会把开票状态设计成状态机。发起开票后先是 PROCESSING，第三方回调成功就更新 SUCCESS，失败就更新 FAILED。如果调用超时或回调没收到，就标记 UNKNOWN，再由定时任务主动查询第三方状态。回调接口要先验签，再根据外部流水号查本地记录。更新状态时要带当前状态条件，比如只有 PROCESSING 或 UNKNOWN 才能改成 SUCCESS，避免重复回调或乱序回调把最终状态覆盖错。

Q66: 如果接口突然变慢，你如何现场排查？

现场排查清单：

1. 看现象
|
+--> 是单个接口慢，还是全站慢？
+--> 是所有用户慢，还是某个租户慢？
2. 看应用
|
+--> CPU / 内存 / GC
+--> Tomcat 线程是否打满
+--> 线程池队列是否堆积
3. 看依赖
|
+--> MySQL 慢 SQL
+--> Redis 是否超时
+--> MQ 是否堆积
+--> 第三方接口是否慢
4. 看日志
|
+--> traceId 串起完整链路
+--> controller/service/mapper 分段耗时
5. 临时止血
|
+--> 限流 / 降级 / 扩容 / 关闭非核心功能

口述答案：

我会先确认慢的范围，是单接口还是整体慢，是所有租户还是某个租户。然后看应用资源，比如 CPU、内存、GC、线程池、Tomcat 线程。接着看依赖，包括 MySQL 慢 SQL、Redis 超时、MQ 堆积、第三方接口耗时。日志里最好有 traceId 和分段耗时，能快速定位慢在哪一层。如果是生产问题，先做止血，比如限流、降级、临时关闭非核心统计接口，再做根因修复。

Q67: 苏州 3-5 年 Java 岗常见项目追问怎么准备？

追问方向	你可以结合的项目	回答重点
Spring Cloud	政企微服务、HDIS	Gateway、Nacos、Feign、配置中心、服务调用
Redis	登录限流、交卷幂等、缓存	SETNX、TTL、Key 设计、缓存一致性
MQ	河南产权异步通知、视频转码、回调削峰	RocketMQ、消费重试、死信、幂等消费
MySQL	报表、治疗记录、库存	索引、事务、乐观锁、执行计划
定时任务	CNRDS、ETL、补偿任务	XXL-Job 分片、可重入、失败补偿

追问方向 你可以结合的项目

第三方接口 发票、保函、HIS

医疗业务 HDIS、CNRDS

回答重点

签名、加密、超时、幂等、状态机

数据质量、字段映射、补报、审计

准备策略：

每个项目至少准备 3 个“问题 + 解决方案 + 结果”：

背景：为什么会有这个问题？

问题：遇到了什么技术/业务难点？

方案：用了什么设计或代码方案？

结果：稳定性/性能/可维护性有什么改善？

反思：如果重做还能怎么优化？

Q68: 项目介绍如何控制在 2 分钟内？

模板：

我最近做的是 xxx 项目，主要解决 xxx 业务问题。

系统使用 Spring Boot / Spring Cloud Alibaba，数据库是 xxx，中间件用到了 Redis / MQ / XXL-Job。

我主要负责 xxx 模块，包括接口设计、表结构设计、核心功能开发和上线问题排查。

其中比较有代表性的点有三个：

第一，xxx，用 xxx 解决了 xxx；

第二，xxx，用 xxx 保证了 xxx；

第三，xxx，通过 xxx 优化了 xxx。

如果您感兴趣，我可以展开讲其中一个细节。

示例：

我最近做的是产权交易中心的业务平台，里面包括专家培训考试、电子发票网关和金融保函等服务。系统用 Spring Boot 和 Spring Cloud Alibaba，网关统一认证，Nacos 做注册配置，Redis 做缓存、限流和幂等。我的职责主要是负责部分业务模块和独立子系统的接口设计、表设计、开发联调和上线问题处理。比较有代表性的点有三个：一个是考试交卷用 Redis SETNX 和数据库唯一约束防重复提交；一个是电子发票通过适配器模式对接不同厂商，并用状态机处理回调；还有一个是视频转码拆成异步任务，避免上传接口被长耗时阻塞。

十三、按简历规则准备的项目闭环与追问链路

这一部分按 `resumeprompt.md` 的要求准备。面试时不要只说“用了 Redis / RocketMQ / Nacos”，必须按下面的链路说清楚：业务场景 -> 问题 -> 方案 -> 为什么不用其他方案 -> 代价 -> 效果。

1. 简历技术栈识别

技术	出现场景	是否保留	面试回答重点
Spring Boot	所有后端业务模块	保留	快速开发 Web API、自动配置、统一异常、参数校验
Spring Cloud Alibaba	河南产权、HDIS 网关/内部调用	保留	Gateway、Nacos、OpenFeign 的业务协同，不夸大成大规模集群
Nacos	服务注册、配置管理	保留	解决服务地址和配置集中管理问题
Gateway	统一入口、认证、上下文透传	保留	统一鉴权、租户上下文、请求日志、路由转发

技术	出现场景	是否保留	面试回答重点
OpenFeign	内部服务调用	保留	声明式 HTTP 调用、统一拦截器透传用户/租户信息
Redis	登录态、黑名单、幂等、限流、缓存	保留	说明 Key 设计、TTL、降级方案
Redisson	河南产权多实例下锁/限流增强	谨慎保留	只在多实例竞争场景说，不用于 HDIS 库存扣减
RocketMQ	河南产权异步通知、转码回调、业务削峰、可靠消息最终一致性	保留	可靠投递、重试、死信、消费幂等、事务消息边界
MyBatis-Plus	业务 CRUD、分页、条件构造	保留	提升开发效率，不替代 SQL 优化能力
MySQL	主业务库	保留	索引、事务、唯一约束、执行计划
MongoDB	第三方响应缓存、开票日志	保留	半结构化报文、TTL 自动清理
Elasticsearch	病历检索/基础搜索	谨慎保留	只说基础检索，不包装复杂搜索平台
XXL-Job	ETL、CNRDS 补报、批量数据同步任务	保留	定时、分片、失败重试、任务日志、可重入设计
Flowable	审批流	保留	只说节点配置和业务流转，不说自研流程引擎
Seata/TCC/Saga	理论和选型边界	了解但不作为项目主线	可说明为什么当前项目选择可靠消息/状态机/补偿，而不是 Seata 强事务
Kafka/Flink/ClickHouse	当前简历不需要	不写	不符合项目主线，容易被追问穿

2. 技术冲突与统一口径

风险点	原因	统一后的说法
Redis SETNX 和 Redisson 混用	面试官会追问到底哪个做分布式锁	SETNX 主要用于接口幂等标记；Redisson 只用于河南产权多实例下需要锁续期/可重入的场景
HDIS 写 RocketMQ 异步	HDIS 按规则是主系统+子系统，单实例为主，不需要包装成复杂 MQ	HDIS 非核心链路用任务状态表、XXL-Job、失败重试和人工补报闭环
HDIS 库存写分布式锁	库存一致性本质在数据库，分布式锁不是首选	使用数据库事务、乐观锁、库存流水、唯一约束
河南产权写微服务	该项目多系统独立部署且通常多实例，允许写微服务组件	重点说 Gateway、Nacos、Redis、RocketMQ、幂等、补偿，不说主导整体架构
分布式事务表达过泛	只说“用了分布式事务”会被追问 Seata/TCC 细节	当前项目主讲本地事务 + RocketMQ 可靠消息 / 状态机 / XXL-Job 补偿；Seata 只做选型对比

3. 河南产权交易中心项目闭环

模块	业务背景	技术问题	方案	为什么不用其他方案	代价/风险	最终效果
统一网关认证	多个业务系统需要统一登录入口	每个服务单独鉴权会重复开发，用户上下文不统一	Gateway + JWT + Redis 黑名单 + 请求头透传	不用 Session：多实例共享成本高；不用各服务重复鉴权：维护成本高	Token 黑名单依赖 Redis，要准备 Redis 异常降级	统一认证入口，子服务只关注业务权限
考试登录限流	开考前用户集中登录、短信验证码集中请求	瞬时请求可能打满登录接口和短信接口	Redis 滑动窗口，按手机号/IP/设备维度限流	不用本地限流：多实例不共享；不用只靠 Nginx：无法按用户维度控制	Redis 不可用时需降级为保守限流或验证码保护	降低峰值冲击和恶意刷接口
交卷幂等	考生重复点击交卷					

模块	业务背景	技术问题	方案	为什么不用其他方案	代价/风险	最终效果
	或网络重试	重复生成成绩、重复写交卷记录	Redis SETNX + 数据库唯一约束 + 状态判断	不只用 Redis: 服务异常可能丢状态; 不前端禁用按钮: 不可靠	需要清理过期幂等 Key, 唯一键冲突要友好返回	保证同一考生同一试卷只处理一次
视频转码	视频上传后需要生成多清晰度文件	FFmpeg 转码耗时, 阻塞上传主流程	上传落任务表, RocketMQ/后台任务异步转码, WebSocket 推送状态	不同步转码: 接口超时; 不用纯定时扫描: 实时性较差	需要处理任务失败、重复执行、转码进度不一致	主流程快速返回, 失败任务可重试
电子发票开票	对接多个税务厂商, 接口格式不同	重复开票、回调乱序、本地状态与第三方状态不一致	本地事务先落库 + RocketMQ 可靠消息 + 状态机 + 状态查询/人工补偿	不用 Seata: 第三方 HTTP 厂商不受本地事务管理; 不直接重复调用: 可能重复开票	状态机规则要严格, UNKNOWN 状态要补偿查询	新增渠道改动小, 状态最终一致、重复开票风险降低
第三方响应缓存	开票查询、厂商响应报文较大且结构不固定	重复查询成本高, 表结构固定不方便扩展	MongoDB 保存响应与短期缓存, TTL 自动清理	不用 MySQL 大字段长期堆积; 不用 Redis 存大报文长期缓存	Mongo 查询条件要建索引, TTL 删除不是实时秒级	降低重复调用, 方便问题追踪

4. HDIS 项目闭环

模块	业务背景	技术问题	方案	为什么不用其他方案	代价/风险	最终效果
网关与租户透传	多院区/多租户使用同一套系统	每个接口手写租户条件容易遗漏	Gateway 解析用户信息, 后端统一处理 tenantId	不靠前端传 tenantId: 可能被篡改; 不每个接口手写: 容易漏	手写 SQL、报表 SQL 要重点审查	降低串租户数据风险
RBAC 权限	医生、护士、管理员权限不同	菜单、按钮、接口权限粒度不同	用户-角色-菜单/按钮/接口权限模型 + Spring Security	不只做前端按钮隐藏: 后端仍可能越权	权限变更要刷新缓存或重新登录	后端能拦截越权操作
库存扣减	药品/耗材按批号、库位、效期管理	并发扣减可能超库存, 重复提交会重复出库	数据库事务 + 乐观锁 + 库存流水 + 唯一业务单号	不用分布式锁: 主要是单业务库一致性, 数据库条件更新更可靠	乐观锁失败要提示重试, 流水要可追溯	保证库存变化可审计、可回滚
CNRDS 上报	国家平台要求上报患者和治疗数据	字段多、编码复杂、错误数据会导致上报失败	字段映射、必填/范围校验、上报日志、失败补报	不直接拼数据上报: 失败后难追踪; 不用复杂分布式事务: 外部平台不可控	映射规则要持续维护, 异常数据要人工处理	上报失败可定位、可补报
ETL 数据集成	HIS/LIS/设备数据同步到 HDIS	外部数据格式不统一, 存在缺失和重复	原始数据落库、清洗转换、状态表、XXL-Job 定时处理	不直接写业务表: 出错难追溯; 不用 MQ 强行实时: 医疗数据允许批处理	任务要可重入, 失败状态要可重跑	数据同步更可追踪、可补偿
慢 SQL 优化	透析记录、费用报表查询频繁	多租户+时间范围+患者条件查询慢	EXPLAIN 分析、联合索引、减少无效 JOIN、分页优化	不一上来加缓存: 先确认 SQL 和索引; 不盲目加索引: 影响写入	索引要结合真实查询条件维护	常见慢查询从秒级降到几百毫秒

5. 面试官追问链路

技术点	第一层: 项目介绍	第二层: 为什么用	第三层: 源码/原理追问	第四层: 架构追问
Gateway	多系统统一入口, 做鉴权和上下文透传	避免每个服务重复处理登录和租户信息	GlobalFilter 执行顺序、路由匹配、请求头透传	Gateway 挂了怎么办? 如何灰度? 如何限流?

技术点	第一层：项目介绍	第二层：为什么用	第三层：源码/原理追问	第四层：架构追问
Nacos	服务注册发现和配置管理	多服务地址和配置集中管理	服务注册心跳、配置监听、长轮询	Nacos 挂了服务还能不能调用？本地缓存怎么工作？
OpenFeign	内部服务间 HTTP 调用	声明式调用，代码更清晰	动态代理、RequestTemplate、负载均衡、拦截器	Feign 超时怎么配？调用失败怎么降级？
Redis 幂等	防止重复交卷、重复开票	快速判断同一业务流水是否处理中	SETNX + EXPIRE 原子性、Lua、Key 过期	Redis 挂了怎么办？为什么还要数据库唯一约束？
Redisson	河南产权多实例下锁续期和可重入	比手写 SETNX 更完整	WatchDog、可重入、锁释放 Lua	锁过期怎么办？为什么不用数据库锁？
RocketMQ	异步通知、转码任务、削峰	长耗时或非核心链路不阻塞主流程	Topic、MessageQueue、ConsumerGroup、重试、死信	MQ 堆积怎么办？消息重复怎么办？如何补偿？
MySQL 事务	库存扣减、状态流转	保证核心业务数据一致	隔离级别、MVCC、行锁、乐观锁	数据库瓶颈怎么办？如何拆查询和写入？
XXL-Job	ETL、CNRDS 补报、批量数据同步任务	批处理可调度、可查看日志、可重试	分片参数、执行器注册、失败重试、阻塞策略	任务跑一半失败怎么办？如何设计可重入？如何避免重复补偿？

6. 高风险问题回答模板

Q: 你为什么 HDIS 不用 RocketMQ?

答：HDIS 里核心业务主要是单系统内的患者、排班、处方、库存、上报流程，数据一致性优先级比削峰更高。像库存扣减这种核心数据，我更倾向用数据库事务、乐观锁和库存流水保证一致性；像 CNRDS 上报和 ETL 同步，失败后需要可追踪、可补报，所以用状态表 + XXL-Job 更直观。MQ 适合河南产权这种多实例、异步通知、视频转码、回调削峰场景，不能为了显得高级所有项目都加 MQ。

Q: Redis SETNX 和 Redisson 到底怎么区分？

答：SETNX 我主要用于短时间幂等标记，比如交卷、开票流水号，目标是快速拒绝重复请求，后面还有数据库唯一约束兜底。Redisson 更适合真正多实例竞争共享资源的场景，因为它有 WatchDog、可重入和 Lua 解锁，自己手写容易漏续期和误删。简历里不会把两者都说成同一个“分布式锁”。

Q: 为什么电子发票用 MongoDB?

答：第三方开票厂商的请求/响应字段变化比较多，且有些报文体积较大。如果全部塞 MySQL 大字段，后续扩展和清理成本高。MongoDB 更适合保存半结构化报文和短期查询缓存，TTL 索引可以自动清理临时数据。但核心业务状态还是落 MySQL，不能把 MongoDB 当主交易库。

Q: 如果河南产权考试登录 QPS 翻 10 倍怎么办？

答：先区分是登录、短信、静态资源还是交卷接口。登录和短信先做 Redis 多维限流，短信接口加降级和验证码保护；交卷接口做幂等和数据库唯一约束；视频、通知等长耗时链路异步化。再看应用线程、数据库连接池、Redis 慢命令和网关限流。如果只是扩机器但数据库已经瓶颈，效果不会好，必须先定位瓶颈层。

Q: MQ 堆积怎么处理？

答：先看 ConsumerGroup 是否在线、堆积量、消费 TPS 和失败重试。如果消费者在线但堆积增长，说明消费能力不足，要看单条消息耗时、SQL、第三方接口和线程池。短期可以扩容消费者、提高并发、限流生产者；长期要优化消费逻辑、批量写库、拆分 Topic。失败消息反复重试时要及时进入死信或异常表，不能让坏消息拖慢正常消息。

7. 分布式事务和分布式调度的高级表达模板

Q: 你项目里到底有没有分布式事务？

答：有处理跨系统最终一致性问题，但不是所有场景都用 Seata。比如电子发票和金融保函涉及本地业务系统、第三方厂商接口、异步回调和状态查询，本地数据库事务无法直接控制第三方 HTTP 接口，所以我们采用本地事务先落库、RocketMQ 可靠消息异步推进、业务状态机控制状态流转、人工查询或状态查询补偿 UNKNOWN/PROCESSING 数据的方案。这个属于最终一致性分布式事务处理，比强行说 Seata 更符合外部接口场景。

Q: 为什么不用 Seata AT？

答：Seata AT 适合我们能控制的多个服务、多个数据库之间的本地事务协调，比如订单库、库存库、账户库都在内部系统里，并且能维护 undo_log。电子发票、金融保函这种第三方 HTTP 厂商接口不受 Seata 管理，厂商已经开票或承保后，本地回滚也不能让对方回滚。所以这种场景要靠状态机、查询补偿、幂等和人工兜底，而不是 Seata 强事务。

Q: 你怎么设计可靠消息最终一致性？

```
业务请求进入
|
v
本地事务：写业务表 + 写消息/状态记录
|
v
提交成功后发送 RocketMQ 消息
|
+--> 消费成功：更新业务状态 / 写处理流水
|
+--> 消费失败：RocketMQ 重试 -> 死信/异常表
|
v
状态查询 / 人工补偿处理 PROCESSING / UNKNOWN
|
v
查询第三方最终状态并补偿本地状态
```

回答重点：本地事务保证“本地业务记录和待处理状态”一致；RocketMQ 负责异步推进；消费端用业务单号幂等；状态查询和人工处理负责兜底补偿；最终状态通过第三方查询接口确认。

Q: 分布式调度你怎么讲得更像真实项目？

答：不要只说“用了 XXL-Job”。要说任务为什么需要调度，比如 CNRDS 批量补报、HIS/LIS 数据同步、历史数据修复。任务表里要有状态字段，任务执行时按批次扫描 WAITING/FAILED/UNKNOWN 数据，处理前把状态改为 PROCESSING，成功改 SUCCESS，失败记录原因和重试次数。任务必须可重入，重复执行不能重复上报或重复改状态。

十四、JVM 与类加载机制

内容来源：JVM.md（已合并）

Q69: 类的生命周期有哪几个阶段？

类的生命周期分为**加载、验证、准备、解析、初始化、使用、卸载**七个阶段：

- 加载：读取字节码生成 Class 对象

- 验证：确保字节码格式安全
- 准备：为静态变量分配内存并赋零值
- 解析：将符号引用转为直接引用
- 初始化：执行 `<clinit>`
- 使用 / 卸载：最终被 GC 卸载

加载 → 验证 → 准备 → 解析 → 初始化 → 使用 → 卸载

Q70: 类加载机制与双亲委派

三层类加载器 + 自定义：

- **启动类加载器** (Bootstrap, C++ 实现)：加载 `<JAVA_HOME>/lib` 核心类 (如 `rt.jar`)，是其他加载器的最终父委托
- **扩展类加载器** (Extension)：加载 `java.ext.dirs` 下的类
- **应用类加载器** (Application)：加载 `classpath` 下用户类
- **自定义类加载器**：继承 `ClassLoader`，可打破双亲委派实现隔离或热部署

双亲委派：保证类加载唯一、安全。

破坏双亲委派的典型案例：

- **Java SPI (JDBC)**：核心接口由启动类加载器加载，实现类由第三方提供，通过**线程上下文类加载器**加载实现类
- **OSGi / Tomcat**：模块化和类隔离，自定义类加载器平级依赖
- **热部署**：用新类加载器重新加载类，旧类加载器连同类一起被 GC

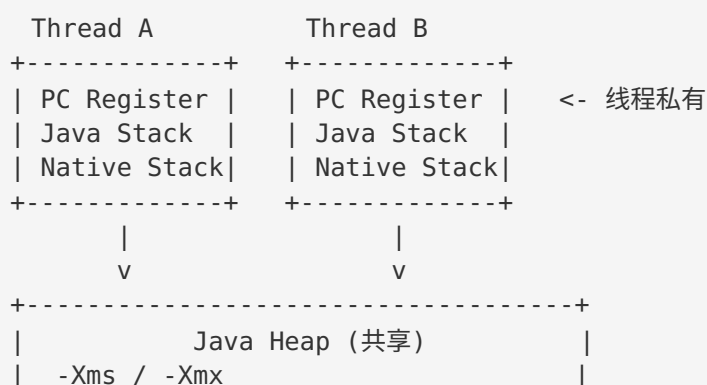
Q71: JVM 内存区域

线程私有：

- **程序计数器**：记录当前线程执行的字节码行号，无 OOM
- **Java 虚拟机栈**：方法栈帧 (局部变量表、操作数栈、动态链接、返回地址)，会抛 `StackOverflowError` / `OutOfMemoryError`
- **本地方法栈**：为 Native 方法服务，类似虚拟机栈

线程共享：

- **Java 堆**：存放对象实例，GC 主战场，分新生代 (Eden + S0/S1) 和老年代，`-Xms` / `-Xmx`
- **方法区 (元空间)**：JDK8+ 用本地内存实现 (Metaspace, `-XX:MaxMetaspaceSize`)，存类信息、常量、静态变量、JIT 代码
- **运行时常量池**：class 文件常量池的运行时表示，存字面量和符号引用
- **直接内存** (非运行时数据区)：NIO 的 `DirectBuffer` 使用，受 `-XX:MaxDirectMemorySize` 限制



```

| +-----+-----+ |
| | Young Gen | Old Gen | |
| | Eden S0 S1| (Tenured) | |
| +-----+-----+ |
+-----+
+-----+
| Method Area / Metaspace (共享) |
| -XX:MaxMetaspaceSize (JDK8+) |
+-----+
+-----+
| Direct Memory (NI0, 本地内存) |
+-----+

```

十五、面试真题补遗 (interview.txt 朋友面经)

内容来源：interview.txt（已合并）。朋友的面试真题绝大部分已覆盖在本文件对应章节，下表给出“真题 → 答案位置”对照；末尾补充 4 个原文档未展开的点。

真题对照表

面试真题	答案位置
Spring Boot Cloud 微服务	三、Spring Cloud 微服务 (Q19)
Redis 分布式锁 / MQ	五、Redis (Q28) / 六、MQ (Q31~Q33)
多线程 / 线程池	一 (Q4 / Q8 / Q9)
SQL 调优 / 慢 SQL	四 (Q25) / Q59
联合索引失效、破坏最左匹配	Q22
IN 也会导致不走索引	Q22
HashMap 扩容原理 / 放 1000 条扩容几次	Q1
int(10) 和 int(11) 区别	Q23
联合索引 a>20 and b=1 order by c	Q22 追问
String 底层 / a == b	Q2
int vs Integer (100 == 100)	Q3
MySQL 底层 B+ 树	Q20
run() vs start()	Q4
单例模式手写	Q5
@Transactional 失效的 80% 原因	Q13
Spring Bean 是否线程安全	Q12
MyBatis 一对多怎么查	Q34
回表	Q21
Spring Security 认证授权	Q17
OpenFeign 跨服务注解与底层	Q18
Redisson 实现报告生成	Q29
SQL 数据量大有索引仍卡	Q25
Spring 事务执行机制	Q14
看门狗因网络问题失效	Q28
接口调用失败重试机制	Q39
Feign 调用失败排查	Q18
WPS 在线编辑接口	Q43

面试真题	答案位置
Activiti/Flowable 工作流表	Q44
一个接口很慢怎么优化	Q35
MQ 消息堆积怎么办	Q31
100w 用户积分每年 1.1 清零	Q38
POI 导入数据量过大内存溢出	Q37
MQ 重复消费与消费成功保证	Q32
死锁怎么造成、怎么避免	Q10
Arthas trace 排查接口慢	Q41
接口安全验证 / BCryptPasswordEncoder	Q40 / Q17
@SpringBootApplication 注解	Q16
分库分表 / 高并发	Q27 / Q36
List 转 Map	Q7
接口要求 QPS 200 怎么做	Q36
缓存不一致（失效/双写模式）	Q30
Spring Boot jar 为什么是 Web 应用	Q45
ORDER BY 排序索引失效	Q24
多线程怎么保证线程安全	Q8
导出文件名 LocalDateTime 线程安全	Q11
线程池核心线程数设计	Q9
千万条数据查询	Q26
Linux 前端调不到后端 Controller	Q42
远程调用每次都发 HTTP 请求建连接	下方补充 4
每次调用都跟 MySQL 建立连接	下方补充 3
IOC 控制反转 / 依赖注入	下方补充 1
消息大量 unack	下方补充 2

补充 1：IOC 控制反转与依赖注入

控制反转（IOC）：对象创建和依赖关系的控制权从代码内部反转给容器（Spring 容器统一管理 Bean 的创建、生命周期和依赖装配）。

依赖注入（DI）：容器在创建 Bean 时把依赖自动注入（构造器注入、Setter 注入、字段注入/
@Autowired），调用方不自己 new。

面试话术：Spring 容器启动时扫描 Bean 定义，通过构造器/Setter/字段注入完成依赖装配；Bean 之间只依赖接口，替换实现不需要改调用方，便于单元测试和扩展。@Autowired 默认按类型注入，多个实现时用 @Qualifier 或 @Primary 区分。

补充 2：RabbitMQ 消息大量 unack 怎么排查

unack（未确认）堆积说明消费者拿了消息但一直没 ack/nack：

1. 看消费者是否在线、是否卡死（线程池满、死循环、阻塞等待）。
2. 看单条消息处理耗时：慢 SQL、第三方接口超时会拖住消费线程。
3. 看 prefetch（basicQos）是否设置过大：一次性预取太多，消息全部变 unack。
4. 看是否有消息处理失败但既不 nack 也不 ack（异常被吞掉）→ 会一直 unack 占着。
5. 处理：先修消费逻辑/加大消费并发；异常必须在 finally 里 nack 或进死信；控制 prefetch 数量。

补充 3：每次调用都跟 MySQL 建立连接怎么办

用连接池（Druid / HikariCP）：应用启动时初始化一批连接，请求复用、用完归还，避免频繁 TCP 握手 + MySQL 认证的开销。配置要点：初始连接数、最大连接数、最小空闲、连接超时、最大等待时间；监控连接池活跃数，防止连接泄漏（用完未归还）打满连接池。

补充 4：远程调用每次都发 HTTP 请求并重新建连接怎么办

用 HTTP 连接池复用 TCP 连接：

- **Feign**：默认 `HttpURLConnection` 无连接池，替换为 `Apache HttpClient / OkHttp`（`feign.httpclient.enabled=true`），配置连接池大小、`maxConnectionsPerRoute`、连接/读取超时。
- HTTP/1.1 默认 `Connection: keep-alive`，服务端（Tomcat）长连接数量也要配套调优。

十六、Java 项目面试补遗（java_project_qa.md 独特内容）

内容来源：java_project_qa.md（已合并）。与原文档重复的题（自动装配、组件表、Gateway 鉴权、Feign 超时、滑动窗口、延迟双删、缓存穿透/击穿/雪崩、Redisson、RabbitMQ 可靠、MQ 选型、接口幂等、视频转码、反问）已并入上文对应章节，以下为原文档未覆盖的补充题。

Q72: Nacos 作为注册/配置中心，和 Eureka/Apollo 有什么区别？

- **Nacos vs Eureka**：Nacos 支持 AP/CP 切换，Eureka 仅 AP；Nacos 自带配置管理，Eureka 不支持；Nacos 支持权重路由、健康检查更灵敏。
- **Nacos vs Apollo**：Nacos 轻量、部署简单、注册+配置一体化；Apollo 功能更丰富（灰度、审计）但更重。中小项目 Nacos 更合适。

Q73: Sentinel 熔断降级策略有哪些？和 Hystrix 区别？

Sentinel 三种熔断策略：

1. **慢调用比例**：RT 超过阈值的比例达到阈值时熔断
2. **异常比例**：异常比例达阈值时熔断
3. **异常数**：异常数达阈值时熔断

Sentinel vs Hystrix：

- Sentinel 控制粒度更细（QPS、线程数、关联、链路），Hystrix 主要是线程池隔离 + 信号量
- Sentinel 有控制台实时监控，规则可持久化到 Nacos；Hystrix 需 Turbine 聚合，配置相对固定

Q74: MyBatis-Plus 多租户怎么实现？

通过 `TenantLineInnerInterceptor` 自动拦截 SQL，所有 SELECT/INSERT/UPDATE/DELETE 自动追加 `tenant_id` 条件：

```
@Bean
public MybatisPlusInterceptor mybatisPlusInterceptor(TenantManager tenantManager) {
    MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();
    interceptor.addInnerInterceptor(new TenantLineInnerInterceptor(
        () -> new LongValue(tenantManager.getCurrentTenantId())
    ));
    return interceptor;
}
```

特定表可配置忽略租户过滤：@InterceptorIgnore(tenantLine = "true")。

Q75: 深度分页怎么优化？

LIMIT 100000, 10 会扫描 100010 行再丢弃前 100000 行。

方案一：游标分页（推荐）

```
SELECT * FROM exam_record WHERE id > 100000 ORDER BY id LIMIT 10;
```

方案二：子查询优化

```
SELECT * FROM exam_record  
WHERE id >= (SELECT id FROM exam_record ORDER BY id LIMIT 100000, 1)  
LIMIT 10;
```

Q76: MyBatis-Plus 和 JPA 怎么选？

维度	MyBatis-Plus	JPA
SQL 控制	完全控制	框架生成
学习曲线	低	中
复杂查询	灵活（报表/统计）	需手写 JPQL/Criteria
适合场景	复杂业务、报表多	简单 CRUD、领域驱动

医疗系统复杂查询多（报表、统计），选 MyBatis-Plus 更灵活。

Q77: 雪花算法 ID 重复怎么办？

雪花算法结构：1 位符号 + 41 位时间戳 + 10 位机器 ID + 12 位序列号。

冲突场景：时钟回拨、机器 ID 重复。解决方案：

1. 机器 ID 由 Nacos 注册中心分配，避免重复
2. 时钟回拨检测：回拨 < 5ms 等待，≥ 5ms 抛异常
3. MyBatis-Plus IdWorker 已内置回拨处理；配置 id-type: assign_id

Q78: 大文件分片上传怎么做（Java）？

1. 前端切片 File.slice() + 计算文件 MD5，服务端秒传判断
2. 并行上传分片，支持断点续传
3. 全部分片上传完成后服务端合并

```
@PostMapping("/upload/chunk")  
public Result uploadChunk(@RequestParam String fileId,  
                           @RequestParam int chunkIndex,  
                           @RequestParam MultipartFile file) {  
    chunkService.saveChunk(fileId, chunkIndex, file);  
    return Result.success();  
}  
  
@PostMapping("/upload/merge")  
public Result mergeChunks(@RequestParam String fileId,  
                           @RequestParam String fileName) {  
    chunkService.mergeChunks(fileId, fileName);  
}
```



```
        return Result.success();
    }
```

Q79: 工厂模式 + 策略模式在项目里怎么用 (Java) ?

```
// 工厂 + Spring 注入: Map<String, InvoiceClient> 自动收集所有实现类
@Service
public class InvoiceClientFactory {
    private final Map<String, InvoiceClient> clients;

    public InvoiceClientFactory(Map<String, InvoiceClient> clients) {
        this.clients = clients;
    }

    public InvoiceClient getClient(String vendor) {
        return Optional.ofNullable(clients.get(vendor))
            .orElseThrow(() -> new BusinessException("不支持的渠道: " + vendor));
    }
}
```

```
// 策略: 保函风控按资方选择不同策略
public interface RiskStrategy {
    RiskResult evaluate(GuaranteeRequest request);
}

@Component("bankA")
public class BankARiskStrategy implements RiskStrategy { /* ... */ }

@Component("bankB")
public class BankBRiskStrategy implements RiskStrategy { /* ... */ }
```

十七、Java 项目讲稿 (口述参考)

内容来源: project_interview_engineering.md (原样并入, 作为项目介绍的口述稿; 与第十二、十三节的表格化答案配合使用)。

面试官到底想听什么

技术面问你项目, 不是为了听你讲业务说明书。他想搞明白三件事:

1. 这系统到底是干嘛的 — 一句话能说清楚, 别绕
2. 你做了什么技术决策 — 为什么这么设计, 不那样设计
3. 你真的动手做过 — 踩过什么坑, 怎么爬出来的

说白了, 面试官不关心"系统有几个模块", 他关心"你遇到问题怎么想的、怎么解决的"。

最让面试官烦的讲法: 背稿、只讲功能不讲技术、问到细节就卡壳、从头到尾没说过一次"为什么这么做"。

介绍项目怎么讲

先说什么 怎么说

这系统干嘛的 — 一句话, 谁用、解决啥问题

你负责什么 你做了什么模块, 什么角色

先说什么 怎么说

你怎么做的 核心方案、技术选型、为什么这么选
结果或反思 有数据说数据，没数据说“现在回头看”

关键技巧：核心方案要主动讲出来，别等面试官问。技术细节不要一开始全倒出来，点一下即可——面试官有兴趣自然会追问，那时候你就有备而来。

总开场白

面试官让你介绍项目的时候，先花 30 秒给个总览，让他知道你主要做什么方向，然后挑一个你最想讲的往下深入。

我主要做两个方向，一个是政企服务平台，一个是医疗信息化系统。

政企这边是在河南产权交易中心做的，包括专家培训考试、电子发票网关、金融保函，还有一些数据可视化和 CMS 的小系统。架构上用的是 Spring Cloud Alibaba 那一套——Gateway 统一入口做登录认证，Nacos 管理注册和配置，业务模块各自独立部署。

医疗那边是 HDIS 血液透析信息系统，在苏州华墨做的，覆盖血透中心从患者建档、排班、处方、治疗、结算到国家平台上报的全流程，配套数据集成和上报的子系统。

我这几年主要是做后端业务模块，接口设计、数据库设计、核心代码开发、联调、上线问题排查都参与。

你想先听哪个？

一、河南产权交易中心微服务业务平台

这个平台其实不是一个单体系统，是产权交易中心内部的一组业务服务。每个模块业务边界不一样，对接的外部系统也不一样。所以分了几个独立的子服务部署。

架构上就是 Gateway 挡在最前面做统一认证，后面跟了几个业务服务——考试、发票、保函、数据可视化。Nacos 管注册发现和配置，服务之间用 Feign 调。

金融保函业务服务

这个最值得讲，对接的保险公司是大家财险。

先说什么业务（一句话版本）：

说白了，有人在 e 交易平台上竞标，需要一份保函。交易平台把这个申请推给我们，我们去调保险公司的接口办保函，办好之后再把结果传回去。

业务流程是这样的：

e 交易平台发起保函申请 → 我们系统接收、校验、保存 → 调大家财险接口提交承保 → 保险公司受理后，我们等回调或者主动查询结果 → 出函成功之后，把保函编号、附件、状态同步回 e 交易平台 → 业务完成。

整个流程里，申请单的状态一直跟着走：待提交 → 已提交 → 承保中 → 已出函 → 已同步。每一步状态变更都有日志。

为什么能用工厂模式统一对接不同保险机构：

虽然现在只接了大家财险一家，但我设计的时候就想了一件事——不管接哪家保险公司，保函的标准业务流程是固定的：提交申请、查询承保状态、接收出函回调、处理退保、下载附件。所以我把这些动作抽象成五个统一接口：

接口	业务含义
保函申请	提交申请人信息、标的信息、保函金额
承保结果查询	主动查询保险机构承保状态
出函回调	保险机构出函后通知我方，更新保函编号和文件
退保/撤销	撤销保函申请或退保
附件下载	下载保函文件、发票等附件

这些接口定义了统一的入参出参模型和各机构的实现契约。大家财险接入时，按照这五个接口实现自己的适配类——报文怎么拼、签名怎么算、状态码怎么映射，全部封装在实现类内部。外部业务层只面对统一接口，完全不用关心底层是哪家保险机构。

我做的技术方案：

核心思路就一条：**保险公司的接口不能参与我们系统的本地数据库事务。** 它可能超时、可能受理了但回调丢了、可能返回中间状态，你没办法用强事务去控制它。

所以我的做法是：

1. 先在自己的数据库里把"业务事实"落下来，也就是先保存申请单和操作日志，状态设为"已提交"。这一步是在本地事务里完成的。保存完之后，这笔业务在我这里就有了依据，无论外部接口怎么挂，我这里是有据可查的。
2. 然后调保险公司接口。接口返回成功，我把状态推到"承保中"；如果超时或者报错，我不会马上判它失败，因为有可能保险公司已经受理了，只是网络回来慢了。我就先记日志，等补偿任务去查。
3. 至于回调，保险公司出函之后会把结果推给我们。我收到回调先验签，然后根据外部流水号找到对应的申请单，再判断能不能更新。这里有个关键的保护逻辑：如果当前状态已经是"已出函"或者"已同步"，那后续来的"失败"回调**不能覆盖成功状态**——因为成功状态是终态，不能被延迟到达的错误消息推翻。
4. 对于那些长时间卡在"已提交"或者"承保中"的申请，我用 XXL-Job 定时任务去主动查保险公司的状态。查到了就更新；查不到或者多次失败，就记失败原因，转人工处理。不会让一笔申请卡在中间状态没人知道。
5. 防重这方面，业务单号和外部流水号都加了数据库唯一索引，重复推送或者重复回调不会产生脏数据。

预判追问——面试官大概率会问这些：

"为什么不直接用 Seata 或者 TCC？"

因为保险公司不是我自己的系统，我没法要求他们改造接口去实现 Try-Confirm-Cancel。而且外部接口本身就是不可靠的，你把它纳入强事务没有任何意义。本地先落库，再通过补偿任务和回调把状态推下去，这种方案才是最务实的。

"补偿任务怎么保证不重复更新？"

每次操作前判断当前状态，终态不可回退。比如已经出函了，补偿任务就不会再把它改成"承保中"。状态流转是单向的。

"如果大家财险的接口格式突然变了怎么办？"

每家保险公司是独立的适配类，格式变化只影响这一家，不会波及其他机构。拿到新文档改对应的报文拼装和结果解析就行。

电子发票系统

一句话：整个河南产权交易中心平台本身就是多组织的，发票是其中一个业务模块。内部多个业务系统要开票，各自对接金蝶、百望太折腾了，所以发票系统统一对接外部厂商。同时开票需要查企业信息，目前对接天眼查，后续可能接企查查。

业务流程：

业务系统发起开票 → 网关校验、保存申请单和明细 → 需要审核的先走审批 → 审核通过后根据租户配置选厂商（金蝶还是百望） → 调厂商接口开票 → 厂商受理后回调返回发票号、发票代码、发票 URL → 我们更新状态和发票信息。同一笔订单可以按明细部分开票，也可以全部开票。已经开过的明细不能再开。如果后续需要红冲，再走红冲申请 → 调厂商红冲接口 → 红冲回调。

整个流程里，每调用一次外部接口都记日志——请求报文、响应报文、耗时、状态变化。为什么？因为发票涉及金额，出了问题必须能追溯到“我们几点几分给厂商发了什么、厂商回了什么”。

为什么能用工厂模式统一对接：

这个模块有两处用了工厂模式。

第一处是**税务厂商**。开票的核心动作是通用的——发票开具、发票查询、发票作废、红冲（申请红字信息表）、查询授信额度。这些动作金蝶和百望都有，但具体实现天差地别：

- **鉴权完全不同：**金蝶走 app_token 换 access_token 的链式鉴权，百望是租户体系加盐值签名
- **接口命名和报文结构不同：**金蝶用 czlx-3（发票开具）、czlx-5（发票查询）这种编号式接口；百望用自己的业务 API 命名
- **红冲逻辑不同：**金蝶专票红冲需要先申请红字信息表再开红票，百望部分票种只支持全额红冲
- **错误码体系不同：**两家的错误码、状态码完全不一样，需要各自映射成我们系统的统一状态

这些差异如果写一堆 if-else 在主流程里，后面加一家就要改一堆。所以抽象成统一接口，一家一个实现类，各家自己的鉴权、拼报文、映射状态码全封装在内部。

第二处是**企业信息查询**——天眼查和后续的企查查，同样的思路。

我做的技术方案：

1. 渠道解耦不说了，这是整个项目最重要的设计前提。
2. 幂等做两层——Redis SETNX 按租户加业务流水号先拦重复提交，数据库唯一索引兜底。Redis 是快速拦截，数据库才是绝对不会漏的网。两层都在入口就做，不让重复请求进入主流程。
3. 异步化——开票主流程是本地事务先落库（申请单、明细、日志），然后直接调厂商接口提交开票申请。厂商是异步处理的，不会同步等出票结果，调完状态标成“开票中”就返回了。等厂商处理完了，调我们这边的回调接口通知结果，验签后更新发票号和状态。如果超时一直没回调，XXL-Job 定时任务扫“开票中”的数据，调厂商查询接口确认最终状态。
4. 企业抬头查询这边也用了一套工厂模式。目前对接的是天眼查，后续可能要接企查查——两家返回的字段结构不完全一样，查询接口的鉴权方式也不一样。所以我抽象了统一的查询接口，天眼查一个实现类，企查查后续一个实现类。业务层只调统一接口，不关心底层查的是哪家。查询回来的数据按租户加请求哈希缓存在 MongoDB 里，半年 TTL，避免重复调用增加成本。
5. 安全——时间戳、摘要签名、SM4 加密，税务系统对接的基本要求。

预判追问：

“如果开票回调一直不来，这单就卡住了吗？”

不会。定时任务会把超过一定时间还处于“开票中”的数据扫出来，主动调厂商查询接口。查到就更新，查不到就记下来告警，让人去看。

"为什么 Redis SETNX 不够，还要数据库唯一索引？"

Redis 可能会重启、数据可能被淘汰。你只依赖 Redis，万一它挂了，重复数据就进来了。数据库唯一索引是最后的兜底，性能不如 Redis 但它绝对可靠。

"为什么用 MongoDB 不用 Redis？"

企业抬头信息不是简单的 key-value，字段很多很杂——税号、开户行、地址、电话，而且不同企业可能缺不同字段。MongoDB 不强制表结构，天然适配。Redis 也能存 JSON，但长期缓存大量杂数据，Redis 成本高多了。

专家培训与考试服务

一句话：产权交易中心的专家在线培训考试平台，从报名、看视频、计时学习、进考试、交卷到审批归档，一条龙。

业务流程：

专家填资料报名 → 报名审批通过 → 后台配好课程、视频和考试计划 → 专家登录后看培训视频 → 前端定时播放心跳，后端记录累计和有效学习时长 → 满足课时要求后进入考试 → 在线答题然后交卷 → 系统判分存成绩 → 需要审批的话走 Flowable 审批流 → 归档

这中间有几个不能出问题的环节——视频上传不能卡住主流程、交卷不能生成多份成绩、集中登录时不能被打垮。

我做的技术方案：

1. 视频上传到七牛云之后，先存记录、标"转码中"，接口直接返回。后台 FFmpeg 异步转码，完了回调通知更新播放地址。上传接口不会被长耗时操作阻塞。视频文件大、网络不稳定的时候，前端走七牛云的分片上传加断点续传。
2. 学习计时防作弊。前端上报播放心跳，中途系统随机弹人机验证。验证没过的那段时间不计入有效时长。有效时长和累计时长分开存，考试资格只看有效时长。
3. 交卷是整个系统并发最高的环节——几千人同时点交卷，算分压力直接砸过来。我做的方案是"同步给反馈、异步干重活"：
 - 请求先进网关，网关转到考试服务的任意一个实例。先走 SETNX——Redis 是共享的，不管请求落到哪个节点，同一个考生 + 试卷 ID 的重复提交在 Redis 层就被拦了。
 - 过了 SETNX 之后保存交卷记录，状态标"已提交"，直接返回"提交成功，判卷中"。这步很快，考生不用等。
 - 然后往 RocketMQ 发消息，消息里带考生 ID 和试卷 ID。消费端是独立部署的判卷服务，从 MQ 拉消息慢慢消化——做答案比对、算分、统计正确率。MQ 在这里的核心作用是削峰——几千人的瞬时算分请求被 MQ 队列摊平，判卷服务按自己能承受的速度逐批处理，数据库不会被瞬时打穿。
 - 判卷完成后，判卷服务更新成绩状态，然后通过 Redis Pub/Sub 广播结果。不管考生连的哪个考试服务节点，WebSocket 都能推过去。
 - 数据库唯一索引兜底，防止 Redis SETNX 失效时重复数据写进去。
4. 视频转码、学习进度统计、成绩通知这些非关键链路也不阻塞主流程，用 RocketMQ 异步或者后台任务消耗掉。
5. 审批流用 Flowable——审批节点、审批人规则、层级流转条件全部走配置。一张业务单据走完审批流程后，审批记录和业务数据分开存，什么时候都能追溯。

预判追问：

"几千人同时交卷，MQ 能撑住吗？"

MQ 本身吞吐量很高，RocketMQ 单机就能扛几十万 QPS。瓶颈不在 MQ，在消费端的算力。消费端控制好并发线程数，逐批从队列拉消息，数据库不会被瞬时打穿。而且消息持久化在 MQ 里，消费端挂了重启之后接着消费，一条消息不会丢。

"交卷后显示"判卷中"，成绩什么时候出来？"

消费端算完就更新成绩状态，前端轮询或者 WebSocket 推送结果。正常几秒内就能出成绩。

"多实例部署下 WebSocket 怎么保证用户能收到推送？"

Redis Pub/Sub。用户连哪个节点不重要，消息通过 Redis 广播到所有节点，当前节点有对应连接就推。

"SETNX 设了过期时间，如果成绩计算花了很长时间怎么办？"

SETNX 只做快速去重判断，进去之后的事务是真正算分的。过期了顶多是短时间内的拦截窗口消失，但数据库唯一索引始终在，重复请求最终还是写不进去。

CMS 存量资产系统

这个就是独立做的一个内容管理系统。业务人员在后台维护资产信息——名称、类型、图片、详情，前台只展示已上架的内容。说白了就是内容管理和展示分离，操作都有日志。

这个项目不大，但我从数据库设计、接口、前后端联调到上线全包了，可以作为独立交付能力的一个例子。

二、HDIS 血液透析信息系统

一句话：医院血液透析中心用的业务系统，从患者建档开始，排班、开处方、治疗、结算、扣库存，最后到国家平台数据上报，是一个业务闭环。

架构上比产权交易中心那边简单一点，不是分布式微服务那种多实例部署，是配套了几个子系统——核心业务服务、数据集成服务、国家平台上报，通过 Nacos 发现，Feign 调接口。

HDIS 核心业务

业务流程：

患者建档 → 排班（排哪天、哪个班次、哪个床位、哪台透析机）→ 接诊 → 医生开处方（透析模式、时长、血流量、抗凝剂这些）→ 上机治疗 → 治疗完成记录（实际超滤量、生命体征）→ 结算费用和扣库存 → 数据进 CNRDS 上报队列

我做的技术方案：

1. 库存扣减——这个场景我有明确的技术选型判断。不用 Redis 锁，用数据库乐观锁。为什么？医疗耗材扣库存不是秒杀，没那个并发量。但它要求绝对的准确性，出错了追溯不了就是医疗事故。我用版本号乐观锁扣减，扣之前校验批号、效期、数量，扣成功之后写库存流水——扣了多少、扣之前多少、扣之后多少、哪个批号、对应哪个业务单号，全部记下来。版本号冲突说明并发冲突了，让业务层重试就行。这套方案吞吐量低一点，但数据绝对不会错。
2. 慢查询和首页性能——透析首页要同时展示患者本次治疗的耗材用量、药品明细、并发症、费用汇总，这些数据分散在多张表里。如果每次打开首页都实时联表查，并发一上来就慢。我的做法是用 RocketMQ 做异步汇总：患者上机后发一条消息，消费端以透析记录 ID 为主键，把耗材、药品、并发症等关联数据查出来，预计算汇总结果，写进一张患者总结表。首页直接查这张总结表，不用联表，查询从原来跨多表的复杂 SQL 变成单表命中。透析记录和费用报表的慢查询也单独做了优化——看执行计划、建联合索引、覆盖索引，典型查询从 1 秒多降到 200 毫秒左右。
3. 多租户隔离——网关登录的时候把租户 ID 放请求头里，核心服务这边 MyBatis-Plus 写了个拦截器从请求头取租户 ID，自动拼到 SQL 条件里。写业务的同事完全不用管"这条数据是哪个医院的"。

预判追问：

"库存为什么不用 Redis 扣库存？"

医疗不是电商。Redis 挂了数据丢了就没法追溯了。这个场景追求的是准确和可追溯，不是极端吞吐量。数据库事务加库存流水是最稳妥的。

"MyBatis-Plus 拦截器具体怎么拿到租户 ID 的？"

网关登录校验 JWT 的时候把 tenantId 取出来，放到一个自定义请求头里。Feign 调核心服务的时候这个头带着走，核心服务这边的拦截器从当前请求上下文里读就行。

面试官如果问"你们项目中哪些地方用了 MQ，哪些没用，为什么？"

这个问题可以从"对方能不能配合你的 MQ"来判断，很好答：

场景	用 MQ？为什么
考试交卷判分	✔️ 用了 判分是自己系统内部的事，收发两端都自己控制，MQ 削峰正好
HDIS 患者上机汇总	✔️ 用了 上机是内部事件，消费者自己查多表算汇总，MQ 解耦
电子发票开票	❌ 没用 金蝶/百望定义了自己的接口和回调方式，中间插 MQ 没意义
金融保函承保	❌ 没用 大家财险同样有自己的回调规范，直接等回调+定时补偿
CNRDS 上报	❌ 没用 定时扫数据直接调接口，XXL-Job 就够了
ETL 数据集成	❌ 没用 同上，定时调度同步

一句话：内部可控的用 MQ 削峰解耦，对接外部系统的走厂商回调+定时补偿。

CNRDS 国家平台上报

一句话：CNRDS 是全国血液净化病例信息登记系统。几十家血透医院要按国家标准上报患者数据——诊断、治疗、用药、检验、透析记录。每家医院本地字段和国家平台标准不完全一样，报失败了要能追踪到"哪家医院、哪条数据、什么原因"。

业务流程：

筛选需要上报的患者和治疗数据 → 从业务表里抽数据组装报文 → 把本地字段映射成国家平台的编码 → 本地校验（必填、格式、范围）→ 分批上报 → 记录成功失败明细 → 失败的标记原因、定时补报

我做的技术方案：

- 1. 批次表和明细表分开。批次表管全局——这次上了多少条、成功多少、失败多少、整体状态是什么。明细表管每一条——上报的报文、返回的结果、失败原因、重试了几次。这样医院运维的人能精确知道哪条数据报失败了、什么原因。
- 2. 校验分层。不上报的数据先做本地校验，字段不全或者不在范围的压根不往上报，记下来让医院修正。过了本地校验的才组报文往上走。
- 3. 补报任务的设计要点是可重入。已经成功的不能重复报，同一批数据执行两遍不能有副作用。失败数据按原因分类——数据本身有问题的通知医院改，网络或平台原因自动重试，超过次数转人工。

ETL 数据集成服务

一句话：医院自己还有 HIS、检验系统、检查系统，这些系统的数据要同步到 HDIS。不同医院的接口五花八门——有 HTTP API、有 WebService 的、有直接开数据库视图让你读的。

业务流程：

按医院配置数据源 → XXL-Job 按医院加数据类型加时间窗口触发同步 → 拉取外部数据 → 单位转换、编码映射、缺失值补全、异常数据拦截 → 按业务唯一键去重 → 写入 HDIS 业务表 → 记录同步日志 → 失败的按窗口重试

我做的技术方案：

- 1. 数据源做成配置化，按医院维护——接口类型、地址、库名表名、字段映射规则全放配置。新增一家医院加一条配置就行，不写死代码。
- 2. 分批同步控制体量。按医院、数据类型、时间窗口分批拉，单批太大容易超时或者内存撑爆，太小又效率低。失败了从指定窗口补拉就行，不影响其他批次。
- 3. 去重靠业务唯一键——医院编码加患者号加检验单号加项目编码加采样时间。外部系统偶尔会重复推数据，已存在的跳过或者更新。

面试官高频追问清单

追问	准备方向
"为什么这么设计，不那样？"	准备好 trade-off（数据库锁 vs Redis 锁、补偿 vs TCC、MongoDB vs Redis）
"最大的难点是什么？"	准备一个具体的、差点翻车但最后解决的故事
"这个是你独立做的吗？"	明确你的角色和贡献边界，协作的功劳也说清楚
"如果再做一次会怎么改？"	每个项目准备一个真诚的反思，不要完美主义
"有没有线上出过问题？"	准备一个排查、定位、修复、预防的完整案例
"你怎么测试？"	能说清楚写了什么测试、验证了什么场景

不同面试选不同项目

面试的公司/方向	优先讲	备选
银行、金融类	金融保函（大家财险）	电子发票网关
医疗、政府类	HDIS 核心 + CNRDS	ETL 数据集成
偏技术和架构的	电子发票网关（工厂 + 策略模式很完整）	金融保函
小公司、全栈方向	CMS + HDIS（覆盖面广）	—

结束语

这几年做下来，最大的变化是思考方式。以前只关心接口能不能通，现在会先想这笔业务从哪来到哪去、中间可能出什么问题。

比如开票、保函、考试交卷、库存扣减这些，功能实现不难，难的是幂等、异常恢复、数据可追溯。所以我现在做业务的习惯是先把流程和状态搞清楚，再设计主表、明细表、日志表、任务表，最后看场景选方案——什么时候用数据库事务就行，什么时候得加 Redis 幂等或者唯一索引，什么时候用 MQ 异步，什么时候用补偿任务或者状态机。

技术方案不是越复杂越好，合适的才是对的。